

UNIVERSIDAD INDOAMÉRICA

FACULTAD DE INGENIERÍAS

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

TEMA:

MODELADO EXPERIMENTAL DE LAS FASES PREFILL Y DECODE DE QWEN3-14B EN NVIDIA A100 MEDIANTE VLLM

Trabajo de Integración Curricular previo a la obtención del título de Ingeniero en Tecnologías de la Información

Autor

José Andrés Ureta Granda

Tutor

Andrés Xavier Rubio Proaño

QUITO – ECUADOR

2026

AUTORIZACIÓN POR PARTE DEL AUTOR PARA LA CONSULTA, REPRODUCCIÓN PARCIAL O TOTAL, Y PUBLICACIÓN ELECTRÓNICA DEL TRABAJO DE INTEGRACIÓN CURRICULAR

Yo, José Andrés Ureta Granda, declaro ser autor del Trabajo de Titulación con el nombre "Modelado experimental de las fases prefill y decode de Qwen3-14B en NVIDIA A100 mediante vLLM", como requisito para optar al grado de Ingeniero en Tecnologías de la Información y autorizo al Sistema de Bibliotecas de la Universidad Tecnológica Indoamérica para que, con fines netamente académicos, divulgue esta obra a través del Repositorio Digital Institucional (RDI-UTI).

Los usuarios del RDI-UTI podrán consultar el contenido de este trabajo en las redes de información del país y del exterior, con las cuales la Universidad tenga convenios. La Universidad Tecnológica Indoamérica no se hace responsable por el plagio o copia del contenido parcial o total de este trabajo.

Del mismo modo, acepto que los Derechos de Autor, Morales y Patrimoniales sobre esta obra serán compartidos entre mi persona y la Universidad Tecnológica Indoamérica, y que no tramitaré la publicación de este trabajo en ningún otro medio sin autorización expresa de la misma. En caso de que exista el potencial de generación de beneficios económicos o patentes, producto de este trabajo, acepto que se deberán firmar convenios específicos adicionales donde se acuerden los términos de adjudicación de dichos beneficios.

Para constancia de esta autorización, en Quito, Ecuador, a los xx días del mes de xx de 2026, firmo conforme:

Autor: José Andrés Ureta Granda

Firma:

Número de Cédula: 1751545185
Dirección: Pichincha, Quito, Belisario Quevedo, La Gasca
Correo Electrónico: andredks334@gmail.com
Teléfono: 0939116995

APROBACIÓN DEL TUTOR

En mi calidad de Tutor del Trabajo de Titulación "MODELADO EXPERIMENTAL DE LAS FASES PREFILL Y DECODE DE QWEN3-14B EN NVIDIA A100 MEDIANTE VLLM", presentado por José Andrés Ureta Granda, para optar por el Título de Ingeniero en Tecnologías de la Información.

CERTIFICO

Que dicho Trabajo de Titulación ha sido revisado en todas sus partes y considero que reúne los requisitos y méritos suficientes para ser sometido a la presentación pública y evaluación por parte del Tribunal revisor que se designe.

Quito, Ecuador, xx de xx de 2026

Andrés Xavier Rubio Proaño

DECLARACIÓN DE AUTENTICIDAD

Yo, José Andrés Ureta Granda, declaro que los contenidos y los resultados obtenidos en el presente Trabajo de Titulación "Modelado experimental de las fases prefill y decode de Qwen3-14B en NVIDIA A100 mediante vLLM", como requerimiento previo para la obtención del Título de Ingeniero en Tecnologías de la Información, son absolutamente originales, auténticos y personales y de exclusiva responsabilidad legal y académica del autor.

Quito, Ecuador, xx de xx de 2026

José Andrés Ureta Granda
(Número de cédula)

APROBACIÓN DE LECTORES

El Trabajo de Titulación ha sido revisado, aprobado y autorizada su impresión y empastado sobre el tema "MODELADO EXPERIMENTAL DE LAS FASES PREFILL Y DECODE DE QWEN3-14B EN NVIDIA A100 MEDIANTE VLLM", presentado por José Andrés Ureta Granda, previo a la obtención del Título de Ingeniero en Tecnologías de la Información, y reúne los requisitos de fondo y forma para su sustentación.

Quito, Ecuador, xx de xx de 2026

Nombres Apellidos
LECTOR

Nombres Apellidos
LECTORA

DEDICATORIA

(Obligatoria)
La dedicatoria va en la parte inferior derecha de la página.

AGRADECIMIENTO

(Obligatorio)
El agradecimiento va en la parte inferior derecha de la página.

Índice de Contenidos

CAPÍTULO I: Introducción **18**

1.1 Contextualización 19
1.2 Estado del Arte 20
1.2.1 Mecanismos de sistema para la inferencia 20
1.2.2 Sistemas de servicio de modelos de lenguaje 21
1.2.3 Síntesis comparativa y posicionamiento 23
1.3 Planteamiento del Problema 25
1.4 Justificación 26
1.5 Objetivos 28
1.5.1 Objetivo General 28
1.5.2 Objetivos Específicos 28
1.6 Alcance del Proyecto 30
1.6.1 Elementos incluidos 30
1.6.2 Elementos excluidos 31
1.7 Fundamentación Teórica 32
1.7.1 Arquitectura Transformer y atención de consultas agrupadas 32
1.7.2 Fases prefill y decode 33
1.7.3 KV cache 34
1.7.4 Gestión de memoria y planificación en vLLM 35
1.7.5 Representación numérica y cuantización 36
1.7.6 Modelo roofline, MFU y MBU 37
1.7.7 Métricas de desempeño de la inferencia 38

CAPÍTULO II: Metodología **39**

2.1 Diagnóstico de la Situación Actual 39
2.2 Área de Estudio 41
2.3 Metodología de Desarrollo o Implementación 43
2.4 Técnicas e Instrumentos de Recolección de Información 48
2.4.1 Triangulación de la separación prefill/decode 48
2.4.2 Telemetría, energía y control de la contención 49

2.5 Requerimientos del Proyecto 51
2.5.1 Requerimientos Funcionales 51
2.5.2 Requerimientos No Funcionales 52
2.6 Diseño de la Solución 52
2.6.1 Conexión con el servidor de inferencia 53
2.6.2 Entorno de software congelado 54
2.6.3 Parche de compatibilidad glibc 55
2.7 Justificación Técnica 57
2.7.1 Versiones y backend de atención 57
2.7.2 Techos teóricos y presupuesto de memoria 58

CAPÍTULO III: Implementación, Pruebas y Resultados 62
3.1 Descripción de la Solución 62
3.2 Proceso de Implementación 63
3.2.1 Piloto con Qwen3-8B: trabajo 30732 63
3.2.2 Servidor único y cliente propio 64
3.2.3 Cambio a Qwen3-14B y diagnóstico de la sesión interactiva 66
3.2.4 Arranque de Qwen3-14B desde el NFS 67
3.2.5 Copia del entorno al disco local 68
3.2.6 Contraste con Ollama 68
3.3 Pruebas y Validación 69
3.3.1 Prueba de humo y verificación del dispositivo 69
3.3.2 Estabilidad de las mediciones 69
3.3.3 Contraste del presupuesto de memoria 70
3.3.4 Validez de las mediciones con Ollama 71
3.4 Resultados 72
3.4.1 Fase de prefill con Qwen3-8B 72
3.4.2 Fase de decode con Qwen3-8B 73
3.4.3 Contraste entre fases 73
3.4.4 Telemetría de la GPU 74
3.4.5 Contraste con Ollama 74
3.4.6 Estado de la campaña con Qwen3-14B y alcance de los resultados 75
3.5 Cronograma y Presupuesto 76

CAPÍTULO IV: Conclusiones y Recomendaciones 77
4.1 Conclusiones 77
4.2 Recomendaciones 77

4.3 Trabajos Futuros 77

REFERENCIAS **78**

GLOSARIO **83**

ANEXO A: Título del Anexo A **84**

Índice de Tablas

1 Comparación de los trabajos revisados en el estado del arte. 24
2 Características de las fases prefill y decode. 34

3 Características de la plataforma experimental. 39
4 Límites de las particiones de Slurm disponibles para el usuario. . . . 41
5 Versión del driver NVIDIA observada en los nodos. 41
6 Área de estudio. 42
7 Iteraciones de prueba y error del trabajo. 44
8 Fases de un trabajo de medición. 46
9 Matriz de la ficha, piloto con Qwen3-8B y barridos preparados para Qwen3-14B. 47
10 Requerimientos funcionales del banco de pruebas. 51
11 Requerimientos no funcionales del banco de pruebas. 52
12 Puntos de acceso de los servidores de inferencia utilizados por el cliente. 54
13 Entorno de software congelado. 55
14 Bibliotecas del entorno con dependencias de glibc 2.32 o superior. . 55
15 Alternativas evaluadas para resolver la incompatibilidad con glibc 2.31. 58
16 Factibilidad en memoria de las configuraciones con Qwen3-14B. . . 61

17 Componentes del banco de pruebas. 62
18 Entorno registrado en el trabajo 30732. 63
19 Duración de las ejecuciones del trabajo 30732. 64
20 Validación del cliente `tfgbench` con Qwen3-0.6B en una partición MIG de 5 GB. 65
21 Experimentos E1 y E2. 66
22 Cronología del arranque de `vllm serve` con Qwen3-14B desde el NFS (2 de noviembre). 67
23 Dispersión de la latencia en las diez iteraciones medidas del piloto con Qwen3-8B. 70

24 Latencia y throughput de prefill de Qwen3-8B con lote 1. 72
25 Comparación de las fases de Qwen3-8B frente a sus techos teóricos ($B = 1$). 74
26 Prefill y decode de `qwen3-coder:30b` servido con Ollama (medianas, 128 tokens de salida). 75
27 Cronograma ejecutado y pendiente. 76

Índice de Figuras

1 Conexión entre el cliente de medición y el servidor de inferencia. . . 53

Índice de Imágenes

Índice de Ecuaciones

1 Atención de producto escalar escalado 33
2 Tamaño del KV cache por token 34
3 Memoria total del KV cache 35
4 Modelo roofline 37
5 Tiempo por token de salida 38
6 Energía estimada por token 38
7 Latencia de prefill por diferencia de latencias 48
8 Throughput de prefill sin servidor 48
9 TPOT por diferencia de latencias 48
10 Energía estimada a partir de la telemetría 50
11 Energía estimada por token generado 50
12 Volumen de los pesos en BF16 59
13 Techo teórico del decode con lote 1 59
14 Techo teórico del prefill 59
15 KV cache por token 60
16 KV cache de la configuración más exigente con Qwen3-8B 60
17 Presupuesto de memoria de la configuración más exigente con Qwen3-8B 60
18 Espacio disponible para el KV cache con Qwen3-14B 60
19 Utilización de cómputo del prefill 72
20 Costo marginal del prefill por token de entrada 72
21 Utilización del ancho de banda en el decode 73

Índice de Anexos

Anexo A: Título del Anexo A 84

UNIVERSIDAD INDOAMÉRICA

FACULTAD DE INGENIERÍAS

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

TEMA: MODELADO EXPERIMENTAL DE LAS FASES PREFILL Y DECODE DE QWEN3-14B EN NVIDIA A100 MEDIANTE VLLM

AUTOR(A): José Andrés Ureta Granda

TUTOR(A): Andrés Xavier Rubio Proaño

RESUMEN EJECUTIVO

[Redacte el resumen del Trabajo de Integración Curricular en un solo párrafo, con un máximo de 300 palabras. Exponga de manera clara el problema, el objetivo general, la metodología empleada, los principales resultados obtenidos y las conclusiones.]

DESCRIPTORES: *[Incluya de 3 a 4 palabras clave representativas, ordenadas alfabéticamente y separadas por comas.]*

UNIVERSIDAD INDOAMÉRICA

FACULTY OF ENGINEERING

BACHELOR'S DEGREE IN INFORMATION TECHNOLOGY ENGINEERING

TOPIC: EXPERIMENTAL MODELING OF THE PREFILL AND DECODE PHASES OF QWEN3-14B ON NVIDIA A100 USING VLLM

AUTHOR: José Andrés Ureta Granda

ADVISOR: Andrés Xavier Rubio Proaño

ABSTRACT

[Write the English version of the executive summary in a single paragraph, with a maximum of 300 words. Clearly present the problem, general objective, methodology, main results, and conclusions.]

KEYWORDS: *[Include 3 to 4 representative keywords in alphabetical order, separated by commas.]*

CAPÍTULO I

INTRODUCCIÓN

La puesta en servicio de un modelo de lenguaje de gran tamaño (LLM, por sus siglas en inglés) obliga a elegir una configuración de inferencia, y esa elección suele apoyarse en métricas agregadas como los tokens por segundo. Estas métricas resumen en una sola cifra dos fases de ejecución, prefill y decode, que someten a la unidad de procesamiento gráfico (GPU) a exigencias distintas [1]. Una cifra agregada registra, por tanto, el resultado de una configuración sin revelar en qué fase se gana o se pierde rendimiento.

El caso de Qwen3-14B [2] sobre una GPU NVIDIA A100 [3], servido mediante el motor de inferencia vLLM [4], vuelve concreta esta limitación. El modelo cabe en una A100 de 40 GB, pero sus pesos ocupan cerca de dos tercios de la memoria, y el espacio restante debe repartirse entre la longitud de contexto y el número de solicitudes simultáneas. En esas condiciones ninguna configuración resulta óptima para todas las cargas de trabajo. Si la configuración adecuada depende de la carga, elegirla exige conocer cómo responde cada fase a los factores que la definen.

De esta tensión surge la pregunta central del capítulo, que la Sección 1.3 formula de forma explícita: cómo influyen la cuantización, la longitud de contexto y la concurrencia en el comportamiento de las fases prefill y decode de Qwen3-14B, ejecutado mediante vLLM sobre una GPU NVIDIA A100, en términos de rendimiento, memoria, energía y calidad. En esta pregunta, la concurrencia designa el número de solicitudes simultáneas. La longitud de contexto comprende la longitud de entrada, o número de tokens del texto de entrada, y la longitud de salida, o número de tokens generados por solicitud.

El recorrido del capítulo sigue el orden en que se construye esa pregunta y su respuesta. La Sección 1.1 sitúa el trabajo en su contexto técnico e institucional, y la Sección 1.2 revisa la literatura para localizar el vacío que deja abierto. La Sección 1.3 expone las consecuencias de ese vacío para el despliegue y las convierte en la pregunta principal. La Sección 1.4 justifica por qué la pregunta merece respuesta, la Sección 1.5 la descompone en una cadena de objetivos en la que cada uno habilita al siguiente y la Sección 1.6 fija el dominio en que esa respuesta será válida. La Sección 1.7 cierra el capítulo con los fundamentos teóricos que el resto de la memoria

requiere.

1.1 Contextualización

Los LLM se han incorporado a asistentes conversacionales, sistemas de consulta documental y flujos de procesamiento automático de texto [5]. Estos modelos se construyen, en su mayoría, sobre la arquitectura Transformer [6]. Una vez entrenado, cada modelo se ejecuta en modo de inferencia ante cada solicitud recibida, de modo que el costo de servirlo se acumula durante toda su vida operativa. La eficiencia de la inferencia condiciona, en consecuencia, la viabilidad de cualquier despliegue.

La inferencia de un LLM se desarrolla en las dos fases ya mencionadas, prefill y decode [1]. La fase prefill procesa el texto de entrada de la solicitud, o prompt, y produce el primer token de salida; la fase decode genera los tokens restantes de uno en uno, de forma autorregresiva. Durante ambas fases, el modelo conserva en la memoria de la GPU las claves y los valores de atención de los tokens previos, estructura que se denomina KV cache. Su funcionamiento se desarrolla en la Sección 1.7; aquí basta señalar que la memoria que ocupa crece con la longitud de contexto y con el número de solicitudes atendidas [4].

Cada fase somete a la GPU a exigencias diferentes, y la configuración de inferencia determina cómo se reparten los recursos entre ambas [1], [7]. En este trabajo, la configuración de inferencia designa la combinación de representación numérica del modelo y parámetros del motor de inferencia con que se atiende una carga de trabajo. De ese reparto dependen dos magnitudes que interesan a quien despliega el servicio: la latencia percibida por los usuarios y el throughput, entendido como el número de tokens procesados por segundo. Una configuración que favorece una de ellas puede perjudicar la otra [7]; de ahí que la distinción entre fases ocupe un lugar central en los trabajos revisados en la Sección 1.2.

El presente trabajo de fin de grado (TFG) se enmarca en el proyecto LLM-CoDesign, que caracteriza la inferencia de modelos de lenguaje considerando conjuntamente el modelo, la carga de trabajo, el hardware y el software. Según la ficha institucional del TFG, el proyecto abarca múltiples modelos, GPU, motores de inferencia y métricas durante cinco años. Un alcance de esa amplitud necesita un primer caso controlado sobre el cual validar la metodología antes de extenderla. Este TFG reduce el alcance del proyecto a ese caso y constituye su primer caso de estudio.

El caso seleccionado combina el modelo Qwen3-14B, perteneciente a la familia Qwen3 [2], con una GPU NVIDIA A100 y el motor de inferencia vLLM [4]. La ficha institucional planteó el caso con Qwen3-8B, el modelo de 8 mil millones de

parámetros de la misma familia. A partir de los primeros ensayos, y por indicación del tutor, el modelo principal pasó a ser Qwen3-14B, y Qwen3-8B se conservó como piloto para calibrar el método. La A100 se comercializa en variantes de 40 GB y 80 GB de memoria [3]; la disponible para este trabajo es la de 40 GB, y con el modelo mayor esa capacidad se convierte en una restricción central del estudio. Fijado el caso, queda por establecer qué sabe ya la literatura sobre la inferencia de LLM en una GPU y qué deja sin resolver para esta combinación.

1.2 Estado del Arte

La distinción entre prefill y decode, presentada en la sección anterior, atraviesa la literatura sobre inferencia de LLM. Esa literatura se concentra en mecanismos de sistema orientados a aprovechar mejor la GPU: la planificación de solicitudes, la gestión de memoria, los kernels de atención, el tratamiento diferenciado de las fases y la cuantización. La Sección 1.2.1 revisa seis trabajos representativos de esas líneas y señala, en cada caso, qué aportan a la pregunta central y qué dejan abierto. La Sección 1.2.2 examina cómo esos mecanismos se integran en los sistemas que sirven los modelos a los usuarios, y la Sección 1.2.3 sintetiza la comparación y sitúa el presente trabajo frente a ella.

1.2.1 Mecanismos de sistema para la inferencia

Yu et al. [8] proponen Orca, un motor de inferencia que planifica la ejecución a nivel de iteración en lugar de a nivel de solicitud. Con esta estrategia, el sistema incorpora solicitudes nuevas al lote y retira las finalizadas en cada paso de generación. Orca reserva, sin embargo, el espacio del KV cache de cada solicitud en bloques contiguos dimensionados de antemano, y esa reserva desperdicia una parte de la memoria disponible [4]. La planificación por iteración mejora así el aprovechamiento del cómputo, mientras deja abierto el de la memoria.

Kwon et al. [4] abordan ese problema con PagedAttention, un mecanismo que gestiona el KV cache en bloques de tamaño fijo, de forma análoga a la memoria virtual paginada de un sistema operativo. La gestión paginada reduce la fragmentación de la memoria y permite compartir bloques entre solicitudes. Sobre este mecanismo, los autores construyen vLLM, que combina la gestión paginada con la planificación a nivel de iteración, conocida como batching continuo. vLLM es el motor de inferencia empleado en este trabajo; su evaluación original, sin embargo, no separa el comportamiento de prefill y decode.

Dao et al. [9] desarrollan FlashAttention, un algoritmo de atención exacta que

reduce los accesos a la memoria de alto ancho de banda de la GPU mediante el procesamiento por bloques. El trabajo pone de relieve el peso del movimiento de datos entre niveles de memoria en el costo de la atención. Esta perspectiva resulta pertinente para el presente TFG, porque el límite que impone la memoria frente al que impone el cómputo es precisamente lo que distingue a una fase de la otra [1].

Agrawal et al. [7] estudian el compromiso entre throughput y latencia y observan que la incorporación del prefill de solicitudes nuevas a un lote interrumpe la generación de las solicitudes que se encuentran en decode. Como respuesta, proponen Sarathi-Serve, un motor de inferencia que divide el prefill en fragmentos (chunked prefill) y los planifica sin detener la generación en curso. La coexistencia de ambas fases en un mismo lote tiene una consecuencia directa para este trabajo: condiciona la manera de medirlas por separado.

Zhong et al. [1] llevan la separación más lejos con DistServe, un motor de inferencia que asigna prefill y decode a GPU distintas para eliminar la interferencia entre fases. Su análisis caracteriza prefill como una fase limitada por el cómputo y decode como una fase limitada por el ancho de banda de memoria. Los autores evalúan cada fase con una métrica propia: el TTFT (*time to first token*), tiempo hasta el primer token, asociado a prefill; y el TPOT (*time per output token*), tiempo por token de salida, asociado a decode. Este trabajo adopta ambas métricas, aunque su caso de una sola GPU no admite la separación física que DistServe propone.

Lin et al. [10] proponen AWQ, un método de cuantización de solo pesos a baja precisión que protege los pesos más relevantes a partir de la distribución de las activaciones. La representación de los pesos con 4 bits (AWQ INT4) reduce la memoria ocupada por el modelo frente al formato de coma flotante de 16 bits BF16. La memoria que libera la cuantización queda disponible para el KV cache, de modo que la representación numérica puede alterar el comportamiento de ambas fases. Por esa razón, este TFG compara las dos representaciones numéricas.

1.2.2 Sistemas de servicio de modelos de lenguaje

Los trabajos anteriores proponen mecanismos que, en la práctica, no operan de forma aislada: se integran en un servidor de inferencia, el sistema que recibe las solicitudes de los usuarios, decide cuándo y con qué recursos se atiende cada una y devuelve los tokens generados. Ejecutar el modelo directamente desde una biblioteca de aprendizaje profundo resuelve el cálculo de una solicitud, pero no las exigencias de latencia y throughput de un servicio que atiende a muchos usuarios; los sistemas de inferencia especializados surgen para cubrir esa distancia [11]. Tres funciones distinguen a un servidor de inferencia de la ejecución directa: la planificación de

las peticiones, el agrupamiento de las solicitudes concurrentes y la gestión de la memoria que ocupa el KV cache.

La planificación determina en qué orden y en qué momento se atiende cada petición. La latencia que percibe un usuario no depende solo del tiempo de cálculo de su solicitud, sino también del tiempo que espera hasta ser atendida y de las solicitudes con las que comparte la GPU. Por ello, la manera en que el servidor planifica y agrupa las peticiones condiciona directamente la latencia del servicio [12]. Esa dependencia explica que el TTFT incluya, en un servidor con carga, el tiempo de espera en cola, como precisa la Sección 1.7.7.

El agrupamiento de solicitudes concurrentes se apoya en la asimetría entre las dos fases: prefill está limitado por el cómputo, mientras que decode está limitado por el ancho de banda de memoria [13]. Como cada paso de decode lee todos los pesos del modelo para producir un solo token por solicitud, reunir varias solicitudes en un mismo paso reparte esa lectura entre ellas. La forma de reunirlas es decisiva. El agrupamiento estático forma un lote y lo mantiene hasta que terminan todas sus solicitudes: las que finalizan antes dejan su espacio sin usar, y las que llegan después esperan al lote siguiente [8]. El agrupamiento continuo, introducido por Orca [8], recompone el lote en cada iteración del modelo, de modo que incorpora solicitudes nuevas en cuanto hay capacidad y retira las finalizadas sin esperar al resto. La contrapartida es que prefill y decode conviven en el mismo lote e interfieren entre sí [7]; el mecanismo que mejora el aprovechamiento de la GPU es, por tanto, el mismo que complica la medición separada de las fases.

Con el agrupamiento continuo, la concurrencia deja de ser un dato fijo de la carga y se convierte en una variable que el planificador gestiona activamente: decide cuántas solicitudes admite, cuáles aplaza y cómo redistribuye los recursos cuando escasean [14]. Para este trabajo, esa gestión obliga a interpretar con cuidado el nivel de concurrencia. El benchmark fija el número de solicitudes simultáneas que se envían al servidor, pero el número de solicitudes que el motor procesa en cada iteración depende de su planificador y de la memoria disponible.

La gestión de la memoria del KV cache es la tercera función, y está acoplada a las dos anteriores. Una reserva contigua dimensionada de antemano, como la de Orca, desperdicia memoria y limita el número de solicitudes que caben en la GPU [4]. La gestión paginada de vLLM asigna el KV cache en bloques de tamaño fijo a medida que crece cada secuencia [4], con el funcionamiento que detalla la Sección 1.7.4. Su consecuencia para el servicio es que la memoria libre, y no una reserva fija por solicitud, determina cuántas solicitudes pueden agruparse. Planificación, agrupamiento y memoria quedan así ligados entre sí, y los tres dependen de la longitud de contexto

y de la concurrencia, dos de los factores que este trabajo hace variar.

vLLM no es el único sistema que combina estas funciones. Entre las alternativas figura SGLang [15], y los propios Sarathi-Serve y DistServe, revisados en la Sección 1.2.1, son motores de inferencia completos. En el ámbito industrial destacan Text Generation Inference (TGI), de Hugging Face, que incorpora agrupamiento continuo y atención paginada [16], y TensorRT-LLM, de NVIDIA, que combina el agrupamiento en vuelo (*in-flight batching*) con un KV cache paginado sobre GPU de NVIDIA [17]. TGI pasó a modo de mantenimiento y su repositorio quedó archivado en marzo de 2026, con la recomendación expresa de migrar a vLLM o SGLang [16]. La elección de vLLM en este trabajo responde a cuatro razones. La primera es que la ficha institucional lo fija como el componente de software S de la configuración experimental. La segunda es que integra en un mismo motor la gestión paginada del KV cache, el agrupamiento continuo y el chunked prefill, que son precisamente los mecanismos cuya interacción con las fases se pretende observar. La tercera es que expone esos mecanismos mediante parámetros de configuración, descritos en la Sección 1.7.4, lo que permite controlarlos y documentarlos. La cuarta es que su código abierto permite fijar y citar la versión exacta empleada, condición de la reproducibilidad. Comparar vLLM con otros motores queda, por su parte, fuera del alcance, como establece la Sección 1.6.2.

1.2.3 Síntesis comparativa y posicionamiento

La Tabla 1 resume, para cada trabajo citado, el sistema o modelo evaluado, el hardware empleado, si separa las fases prefill y decode, las métricas que reporta y su limitación para el presente TFG; su última fila describe el diseño de este trabajo. Ninguno de los trabajos revisados caracteriza la combinación de Qwen3-8B, A100 y vLLM, y todos ellos son anteriores a la publicación de la familia Qwen3 [2].

La última fila de la Tabla 1 plantea una objeción inmediata. DistServe [1] y Splitwise [18] ya separan prefill y decode y ya los miden por separado; Splitwise caracteriza, además, la potencia de cada fase [18]. Cabe preguntarse, entonces, qué añade una nueva caracterización por fases. La respuesta no está en qué fases se miden, sino en dónde se ejecutan. Ambos trabajos desagregan las fases sobre recursos distintos: asignan prefill a unas GPU y decode a otras, y diseñan el sistema para que cada grupo de recursos atienda una sola fase. Esa desagregación es la solución de un operador hiperescalar, que dispone de numerosas GPU interconectadas y puede dedicar una parte de ellas a cada fase. Una universidad o un laboratorio sin acceso a esa infraestructura sirve el modelo, en cambio, desde una única GPU, en la que ambas fases comparten el cómputo, el ancho de banda de memoria y el espacio

Tabla 1: Comparación de los trabajos revisados en el estado del arte.

Trabajo	**Año**	**Sistema o modelo evaluado**	**Hardware**	**¿Separa prefill y decode?**	**Métricas reportadas**	**Limitación para este TFG**
Flash-Attention [9]	2022	Kernel de atención; entrenamiento de BERT y GPT-2	GPU A100	No aplica: optimiza un kernel	Tiempo de ejecución y memoria	Optimiza un cálculo aislado, no el servicio
Orca [8]	2022	Motor Orca; modelos de la familia GPT-3	Varias GPU A100	No: ambas fases en el mismo lote	Latencia y throughput	Reserva contigua del KV cache
Pope et al. [13]	2023	Inferencia de PaLM	TPU v4	Sí, en el análisis: prefill limitado por cómputo, decode por memoria	Latencia y MFU	Otro acelerador; modelo repartido entre muchos chips
vLLM [4]	2023	Motor vLLM; modelos OPT y LLaMA	GPU A100	No	Throughput a latencia normalizada	Evaluación agregada, sin desglose por fases
AWQ [10]	2024	Cuantización de pesos; modelos LLaMA y OPT	GPU de escritorio y embebidas	No	Perplejidad, precisión y aceleración	No mide el efecto por fases bajo carga concurrente
Sarathi-Serve [7]	2024	Motor Sarathi-Serve; varias familias de modelos	GPU A100 y A40	En parte: distingue las fases, pero las mezcla en el lote	TTFT, latencia entre tokens y capacidad	La mezcla de fases condiciona su medición separada
DistServe [1]	2024	Motor DistServe; modelos OPT	Varias GPU A100	Sí: GPU distintas para cada fase	TTFT, TPOT y goodput	Requiere varias GPU
SGLang [15]	2024	Motor SGLang, alternativo a vLLM	GPU A10G y A100	No	Throughput y latencia	Comparar motores queda fuera del alcance
Splitwise [18]	2024	Separación de fases; modelos BLOOM y LLaMA-2	Servidores DGX-A100 y DGX-H100	Sí: máquinas distintas para cada fase	TTFT, latencia entre tokens, throughput, potencia y costo	Requiere un clúster de varias máquinas
Este TFG	**2026**	Qwen3-14B en BF16 y AWQ INT4, con Qwen3-8B como piloto, servido con vLLM; controla longitud de entrada, longitud de salida y concurrencia	Una única GPU A100-SXM4-40GB	Sí: ambas fases comparten una sola GPU	TTFT, TPOT, throughput, latencia total, memoria máxima, utilización, potencia media y energía estimada por token	Validez limitada a Qwen3-14B y Qwen3-8B, A100 y vLLM; energía estimada por telemetría

Elaborado por: José Andrés Ureta Granda
Fuente: Elaboración propia

disponible para el KV cache. En ese régimen, la interferencia que la desagregación elimina forma parte de las condiciones de funcionamiento, y el reparto de recursos entre fases no lo decide la topología del clúster, sino el planificador del motor y sus parámetros de configuración. Una sola GPU compartida no es, por tanto, una versión reducida de un sistema desagregado, sino un régimen distinto, con compromisos propios que los trabajos desagregados no necesitan resolver. Este TFG caracteriza el comportamiento de ambas fases precisamente en ese régimen.

La literatura revisada explica, por tanto, por qué prefill y decode se comportan de forma distinta, pero no ofrece una caracterización experimental de ambas fases cuando comparten una sola GPU, y menos aún para el caso seleccionado. La Sección 1.3 examina las consecuencias de ese vacío para quien debe configurar el servicio.

1.3 Planteamiento del Problema

El vacío identificado en el estado del arte tiene un efecto práctico. La ficha institucional del trabajo constata que la selección de una configuración de inferencia suele basarse en métricas agregadas, como los tokens por segundo, sin distinguir el comportamiento de las fases prefill y decode. Una métrica agregada resume en una sola cifra dos procesos con exigencias distintas sobre la GPU [1]. El resultado es una visión incompleta del sistema, que registra los compromisos entre fases sin explicarlos.

Esa visión incompleta deja sin explicar varios fenómenos relevantes para el despliegue. No permite justificar por qué una configuración mejora el throughput y empeora la latencia interactiva, ni en qué casos la cuantización permite aumentar la concurrencia. Tampoco aclara cómo influye la longitud de entrada sobre el TTFT, ni cómo afecta el crecimiento del KV cache a la latencia durante decode. Cada uno de estos fenómenos se origina en una fase concreta; por ello, ninguna cifra agregada alcanza a explicarlo.

La consecuencia práctica es la dificultad para elegir una configuración adecuada según el uso previsto. Según la ficha institucional, un servicio de chat interactivo, una consulta documental basada en recuperación (RAG, por sus siglas en inglés) y un procesamiento por lotes imponen prioridades diferentes. Una configuración apropiada para un escenario puede resultar, por tanto, inadecuada para otro.

El caso de Qwen3-14B en una A100 de 40 GB ilustra la situación con especial nitidez. El modelo puede ejecutarse en esa GPU, pero esta posibilidad no garantiza la existencia de una configuración universalmente óptima: el rendimiento depende

de la carga de trabajo, de la concurrencia y de la representación numérica del modelo. Con los pesos ocupando la mayor parte de la memoria, además, cada token adicional de contexto y cada solicitud simultánea compiten por un espacio reducido, de modo que una configuración que funciona para un uso puede no caber siquiera en la GPU para otro. Para esta combinación concreta no se dispone, además, de una caracterización experimental que separe ambas fases, como muestra la Tabla 1. Ese vacío, trasladado al despliegue, constituye el problema del trabajo.

La caracterización que falta puede concretarse en preguntas específicas que la vuelven medible. Las dos primeras atañen a cada fase por separado: cómo cambia el TTFT cuando aumenta la longitud de entrada y cómo cambia el TPOT cuando aumenta la concurrencia. La tercera examina cómo influye la longitud de salida en la latencia y en la energía.

Otras tres preguntas se ocupan de la memoria y de la capacidad del sistema: cuánta memoria ocupan las representaciones numéricas BF16 y AWQ INT4, en qué condiciones la cuantización mejora la capacidad del sistema y qué configuraciones producen saturación o errores por falta de memoria. Las dos últimas completan el recorrido: cuáles son los principales cuellos de botella de cada fase y con qué error estima un modelo predictivo básico el TTFT y el TPOT a partir de las variables experimentales delimitadas en la Sección 1.6.1.

Responder a estas preguntas exige una secuencia de productos: un entorno en el que ejecutar el modelo, una carga controlada, un conjunto de mediciones, un análisis por fases y un modelo predictivo. La Sección 1.4 expone por qué estas preguntas merecen respuesta, y la Sección 1.5 organiza esos productos como una cadena de objetivos. Todas ellas convergen en una sola pregunta, con la que se cierra el planteamiento.

En consecuencia, el problema descrito se formula mediante la siguiente pregunta principal: ¿cómo influyen la cuantización, la longitud de contexto y la concurrencia en el comportamiento de las fases prefill y decode de Qwen3-14B, ejecutado mediante vLLM sobre una GPU NVIDIA A100, en términos de rendimiento, memoria, energía y calidad?

1.4 Justificación

La pregunta se justifica, en primer lugar, desde el punto de vista técnico. El análisis separado de prefill y decode permite explicar los compromisos entre latencia, throughput y memoria que las métricas agregadas solo registran. Williams et al. [19] proponen el modelo roofline, que relaciona el rendimiento alcanzable con los límites

de cómputo y de ancho de banda de memoria del procesador. Este marco ofrece una referencia para situar cada fase respecto a los límites de la A100 y sustenta el análisis de cuellos de botella que persigue el objetivo específico OE4 (Sección 1.5.2).

Desde el punto de vista operativo, el trabajo orienta la elección de configuraciones para los tres escenarios de uso presentados en la Sección 1.3. Según la ficha institucional, el chat interactivo prioriza un TTFT y un TPOT bajos; la consulta documental requiere un equilibrio entre longitud de entrada, latencia y memoria; y el procesamiento por lotes prioriza el throughput y la eficiencia energética. Una caracterización por fases permite relacionar cada una de esas prioridades con la fase que la gobierna.

La utilidad del trabajo se materializa en el prototipo Qwen-A100 Profiler, que la ficha institucional define como su producto tecnológico. Esta herramienta automatiza la ejecución de los experimentos, recolecta las métricas, almacena los resultados de forma estructurada y estima el TTFT y el TPOT mediante el modelo predictivo que persigue el objetivo específico OE5. El conjunto de datos experimental y los procedimientos de reproducción completan los entregables reutilizables, de modo que el estudio puede repetirse sin reconstruir la metodología.

Esos entregables definen a los beneficiarios del trabajo. El beneficiario directo es el proyecto LLM-CoDesign, que obtiene su primer caso de estudio y una metodología de caracterización aplicable a otras combinaciones de modelo, hardware y software. Los beneficiarios indirectos son los equipos técnicos que despliegan LLM en infraestructura propia y necesitan criterios cuantitativos para configurar el servicio.

La justificación exige, por último, que el trabajo sea factible. En el plano técnico, Qwen3-14B en BF16 cabe en la memoria de una única A100 de 40 GB: sus pesos ocupan 27,5 GiB, según el registro del propio motor, y dejan un margen estrecho pero suficiente para el KV cache de las configuraciones de menor concurrencia, como cuantifica la Sección 2.7.2. Las herramientas necesarias (vLLM, las herramientas de telemetría NVML y DCGM, y las bibliotecas de análisis de datos) son de libre acceso. El trabajo no requiere entrenar el modelo, desarrollar kernels de GPU ni modificar internamente vLLM. En el plano operativo, el plan de trabajo comprende doce semanas con una dedicación de 15 a 20 horas semanales, y la factibilidad depende de disponer de acceso controlado a la GPU durante las campañas de medición. El piloto con Qwen3-8B ya demostró que la medición por fases es ejecutable en el clúster disponible, y los obstáculos encontrados con el modelo mayor, descritos en el Capítulo II, son de infraestructura y no del método. Justificada la pregunta, la Sección 1.5 la traslada a una cadena de objetivos.

1.5 Objetivos

Las preguntas formuladas en la Sección 1.3 se responden mediante un objetivo general y cuatro objetivos específicos. Sus enunciados reformulan los de la ficha institucional aprobada: conservan su contenido, expresan cada objetivo con un solo verbo e integran el diseño del benchmark en el objetivo de medición, del que es el instrumento.

1.5.1 Objetivo General

Identificar el efecto de la cuantización, la longitud de contexto y la concurrencia sobre el comportamiento de las fases prefill y decode de Qwen3-14B, servido mediante vLLM sobre una única GPU NVIDIA A100, en términos de rendimiento, memoria, energía y calidad, mediante la medición controlada de ambas fases y un modelo predictivo básico del TTFT y del TPOT, como base para la selección de configuraciones de inferencia según el escenario de uso.

El objetivo general traslada la pregunta principal a una acción verificable. Retoma los mismos tres factores y las mismas cuatro dimensiones que la pregunta nombra, precisa los dos medios con que se alcanza, la medición controlada de ambas fases y el modelo predictivo, y declara su finalidad: servir de base a la elección de configuraciones en los escenarios de uso descritos en la Sección 1.3.

1.5.2 Objetivos Específicos

OE1. Levantar un entorno reproducible de ejecución de Qwen3-14B, y de Qwen3-8B como piloto, mediante vLLM sobre una A100, mediante la fijación y documentación de las versiones de CUDA, del controlador de NVIDIA, de PyTorch y de vLLM en un contenedor o entorno versionado, como garantía de que las diferencias observadas entre configuraciones puedan atribuirse a los factores estudiados y no a cambios del entorno.

OE2. Medir el TTFT, el TPOT, el throughput, la memoria GPU, la utilización y el consumo energético estimado, mediante un benchmark parametrizable con niveles controlados de longitud de entrada, longitud de salida y concurrencia sobre las representaciones BF16 y AWQ INT4, como base empírica del análisis por fases.

OE3. Analizar por separado el comportamiento de las fases prefill y decode, mediante el contraste de las mediciones con los límites de cómputo y de ancho de banda de memoria de la A100 según el modelo roofline, con vistas a la

localización de los principales cuellos de botella de cada fase.

OE4. Construir un modelo predictivo básico del TTFT y del TPOT, a partir de las variables experimentales y de las relaciones identificadas en el análisis por fases, como soporte de la estimación y la recomendación de configuraciones del prototipo Qwen-A100 Profiler dentro de los niveles estudiados.

Los cuatro objetivos específicos se leen como una cadena y no como una lista. Los párrafos siguientes explican, para cada uno, qué vacío cierra, por qué condiciona al objetivo que le sigue y qué aporta al objetivo general.

OE1 cierra la ausencia de un entorno controlado en el que medir. La ficha institucional exige congelar y documentar las versiones de CUDA, del controlador de NVIDIA, de PyTorch y de vLLM, de modo que los experimentos puedan reconstruirse. Sin esa condición, ninguna diferencia observada entre configuraciones podría atribuirse a los factores estudiados en lugar de a cambios del entorno. Por esa razón, OE1 es condición necesaria de OE2: dos mediciones solo son comparables si se realizan sobre el mismo entorno. Al objetivo general, OE1 le aporta la reproducibilidad de la medición.

OE2 cierra dos vacíos a la vez: la distancia entre la pregunta y los datos, y la ausencia de mediciones para esta combinación, señalada en la Tabla 1. Las preguntas específicas varían la longitud de entrada, la longitud de salida y la concurrencia, y ningún entorno genera por sí solo cargas con esos niveles controlados. El benchmark parametrizable produce esas cargas de forma sistemática sobre el entorno de OE1; es, por tanto, el instrumento de la medición y no un fin en sí mismo. La cuantización se introduce al ejecutar el modelo en sus dos representaciones numéricas. El TTFT y el TPOT reflejan cada fase por separado; el throughput, la memoria GPU, la utilización y el consumo energético cubren las dimensiones de rendimiento, memoria y energía del objetivo general. El consumo energético se obtiene como estimación a partir de la telemetría interna de la GPU, según precisa la Sección 1.6.1. OE2 es condición necesaria de OE3, porque una métrica solo puede atribuirse a un factor, y a una fase, si la carga que la produjo está controlada.

OE3 cierra el vacío central del planteamiento: la explicación de los compromisos que las métricas agregadas registran sin explicar. Separar prefill y decode permite atribuir cada efecto observado a la fase que lo produce, y el contraste con los límites del modelo roofline, descrito en la Sección 1.7.6, permite localizar en cuál de ellas aparece cada cuello de botella. Es condición necesaria de OE4, porque el modelo predictivo estima el TTFT y el TPOT por separado, y cada estimación exige conocer qué variables gobiernan la fase correspondiente. Con OE3 se completa el primero de los dos medios que nombra el objetivo general.

OE4 cierra el paso de la descripción a la anticipación. Un modelo predictivo básico estima el TTFT y el TPOT de una configuración a partir de las variables experimentales, sin necesidad de ejecutarla, dentro de los niveles estudiados. Como último eslabón, OE4 no habilita otro objetivo específico; alimenta, en cambio, la estimación que realiza el prototipo Qwen-A100 Profiler descrito en la Sección 1.4. Con él se completa el segundo medio del objetivo general.

Para OE4, la ficha institucional establece como meta referencial un error porcentual absoluto medio (MAPE) inferior al 15–20 % en las métricas principales. Esta meta es ajustable después de los experimentos piloto. No constituye, por tanto, un compromiso cerrado sobre la precisión del modelo predictivo.

Leídos en conjunto, los cuatro objetivos forman una cadena: el entorno de OE1 sostiene las mediciones de OE2, que alimentan el análisis por fases de OE3 y, a través de él, el modelo predictivo de OE4. La dimensión de calidad que el objetivo general incluye se aborda mediante la comparación limitada entre BF16 y AWQ INT4 que precisa la Sección 1.6.1. Queda por precisar hasta dónde llega la respuesta, cometido de la Sección 1.6.

1.6 Alcance del Proyecto

La respuesta a la pregunta central vale únicamente dentro de un dominio experimental definido. La Sección 1.6.1 precisa ese dominio y la Sección 1.6.2 enumera lo que queda fuera de él.

1.6.1 Elementos incluidos

La unidad de análisis es la configuración experimental $C = (M, W, H, S)$. En ella, M es el modelo Qwen3-14B; W, las características de la carga de trabajo; H, una GPU NVIDIA A100; y S, vLLM con sus parámetros de configuración. Qwen3-8B interviene como modelo piloto: con él se calibran el método y los instrumentos antes de la campaña principal, y sus resultados permiten contrastar cómo escala cada fase con el tamaño del modelo. La variante de la A100 empleada es la SXM4 de 40 GB, y se documenta como parte del entorno experimental porque su capacidad de memoria condiciona los niveles máximos de longitud de contexto y concurrencia alcanzables.

Sobre esa unidad se varían cuatro factores, que constituyen las variables experimentales. La cuantización se evalúa a través de la representación numérica, con dos niveles: BF16 y AWQ INT4. La longitud de entrada toma cuatro niveles, de 128, 512, 2048 y 8192 tokens; la longitud de salida, dos niveles, de 64 y 256 tokens; y la

concurrencia, cuatro niveles, de 1, 4, 8 y 16 solicitudes simultáneas. La combinación de los niveles produce $2 \times 4 \times 2 \times 4 = 64$ configuraciones, y con cinco repeticiones por configuración la campaña comprende alrededor de 320 ejecuciones medidas. Estos niveles son los de la ficha y se ajustan con los resultados del piloto, como la propia ficha prevé: con Qwen3-14B en la A100 de 40 GB, las combinaciones de contexto largo y concurrencia alta no caben en la memoria disponible, según calcula la Sección 2.7.2, y en ellas se mide el comportamiento del motor en el límite de la memoria. Las configuraciones se agrupan además en tres tipos de carga de trabajo: chat corto, consulta documental y contexto largo.

Las métricas registradas son el TTFT, el TPOT, el throughput, la latencia total de la solicitud, la memoria máxima utilizada, la utilización de la GPU, la potencia media y la energía estimada por token. La energía se presenta como una estimación basada en la telemetría interna de la GPU, obtenida mediante NVML y DCGM. No constituye una medición eléctrica calibrada externamente, salvedad que la ficha institucional exige hacer explícita.

El alcance incluye una comparación limitada de calidad entre BF16 y AWQ INT4 para identificar posibles degradaciones asociadas a la cuantización. Incluye también el prototipo Qwen-A100 Profiler en su versión de producto mínimo viable, con una recomendación básica de configuraciones para los tres escenarios de uso. Definido lo que el trabajo abarca, la Sección 1.6.2 fija lo que excluye.

1.6.2 Elementos excluidos

Las exclusiones siguientes acotan el trabajo a un caso controlado e impiden que sus conclusiones se extiendan más allá de lo medido:

- Entrenamiento completo o fine-tuning del modelo.
- Comparación con otras familias de LLM o con tamaños de Qwen3 distintos de 14B y 8B.
- Evaluación de múltiples GPU y ejecución multi-GPU.
- Paralelismo tensorial o de pipeline.
- Comparación sistemática entre varios motores de inferencia.
- Modelos Mixture-of-Experts o multimodales.
- Speculative decoding.
- Longitudes de entrada superiores a 8192 tokens en la campaña principal.

- Predicción del comportamiento en otro hardware.
- Simulación de sistemas distribuidos.
- Desarrollo completo del asesor multiobjetivo de LLM-CoDesign.

Las mediciones de contraste con el servidor Ollama que recoge el Capítulo II se hicieron con otros modelos, uno de ellos de tipo Mixture-of-Experts, y tienen carácter exploratorio: sirven para contrastar instrumentos y tendencias, y no forman parte de la campaña ni sustentan sus conclusiones. Las conclusiones del trabajo serán válidas únicamente para la combinación de Qwen3-14B, con Qwen3-8B como referencia, NVIDIA A100 y vLLM. No se formularán afirmaciones de superioridad general sobre otras plataformas o modelos. Delimitado así el dominio de la respuesta, la Sección 1.7 expone los fundamentos teóricos que los capítulos siguientes requieren.

1.7 Fundamentación Teórica

Las secciones anteriores emplean conceptos que solo se han mencionado: las fases prefill y decode, el KV cache, la cuantización y los límites de cómputo y de memoria de la GPU. Esta sección los desarrolla en el orden en que intervienen durante la inferencia. Primero se describe la arquitectura del modelo y las dos fases de su ejecución; después, la estructura de memoria que las conecta y los mecanismos con que vLLM la gestiona; por último, la cuantización, el modelo roofline y las métricas con que se caracteriza cada fase. Los valores numéricos para Qwen3-14B, Qwen3-8B y la A100 se calculan en el Capítulo II; aquí se establecen las relaciones de las que esos valores se derivan.

1.7.1 Arquitectura Transformer y atención de consultas agrupadas

Qwen3-14B y Qwen3-8B son modelos Transformer de solo decodificador de la misma familia, que comparten arquitectura y difieren en el número y el tamaño de sus capas [2]. La arquitectura Transformer apila capas idénticas, cada una formada por un bloque de autoatención y una red de alimentación directa [6]. En un modelo de solo decodificador, cada token solo puede atender a los tokens que lo preceden, de modo que el texto se genera de forma autorregresiva: cada token nuevo se calcula a partir de todos los anteriores.

El bloque de autoatención proyecta la representación de cada token en tres vectores: una consulta (Q), una clave (K) y un valor (V). La salida de la atención se obtiene con la Ecuación 1, donde d_k es la dimensión de las claves [6]. El producto $QK^\top$

compara la consulta de cada token con las claves de todos los tokens previos, y el resultado pondera los valores correspondientes. La atención multicabeza repite este cálculo en paralelo con varias proyecciones independientes, denominadas cabezas.

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{1}$$

La Ecuación 1 tiene una consecuencia directa para la inferencia: para generar un token, el modelo necesita las claves y los valores de todos los tokens anteriores. Recalcularlos en cada paso sería redundante, porque no cambian; por ello se conservan en memoria, en la estructura que la Sección 1.7.3 describe.

La atención de consultas agrupadas (GQA, por sus siglas en inglés) reduce el volumen de esa estructura. En lugar de asignar claves y valores propios a cada cabeza de consulta, GQA reparte las cabezas de consulta en grupos que comparten un mismo par de cabezas de clave y valor [20]. Ambos modelos emplean GQA con 8 cabezas de clave y valor: Qwen3-14B las combina con 40 cabezas de consulta y Qwen3-8B, con 32 [2]. Almacenan, por tanto, cinco y cuatro veces menos claves y valores, respectivamente, que un modelo equivalente con atención multicabeza convencional. Esta reducción es la que hace manejable el KV cache en una única GPU con contextos largos, y resulta decisiva en Qwen3-14B, cuyos pesos dejan poca memoria libre.

1.7.2 Fases prefill y decode

La inferencia de una solicitud se divide en dos fases con un patrón de cálculo distinto [1]. En la fase prefill, el modelo procesa los L_{in} tokens del prompt en una sola pasada. Como todos ellos se conocen de antemano, se calculan en paralelo: cada capa opera sobre una matriz de L_{in} filas, y los pesos leídos desde la memoria se reutilizan para todos los tokens. La fase termina con el primer token de salida y con las claves y los valores del prompt ya almacenados.

En la fase decode, el modelo genera los $L_{out} - 1$ tokens restantes, uno por paso. Cada paso procesa un único token por solicitud, porque el siguiente depende de él. El paso exige, sin embargo, leer todos los pesos del modelo y las claves y los valores acumulados, aunque el cálculo realizado con ellos sea pequeño. Cuando el motor agrupa B solicitudes en decode, un mismo paso reutiliza los pesos para las B solicitudes, pero cada una aporta un solo token.

La diferencia entre ambas fases se expresa mediante la intensidad aritmética, definida en la Sección 1.7.6 como el número de operaciones realizadas por byte leído

de memoria. En las capas lineales, un modelo de N parámetros realiza aproximadamente $2N$ operaciones de coma flotante por token, una multiplicación y una suma por parámetro. En BF16 lee $2N$ bytes de pesos por pasada. La intensidad aritmética del prefill crece, por tanto, de forma aproximadamente proporcional a L_{in}, mientras que la del decode crece con el número de solicitudes B del lote. Por esa razón, el prefill tiende a quedar limitado por el cómputo y el decode por el ancho de banda de memoria [1]. La Tabla 2 resume las diferencias.

Tabla 2: Características de las fases prefill y decode.

Característica	**Prefill**	**Decode**
Tokens procesados por solicitud	Los L_{in} del prompt, en una pasada	Uno por paso, durante $L_{out} - 1$ pasos
Intensidad aritmética	Crece con L_{in}	Crece con el tamaño del lote B
Recurso limitante típico	Cómputo	Ancho de banda de memoria
Efecto sobre el KV cache	Lo crea para el prompt	Lo amplía en un token por paso
Métrica asociada	TTFT	TPOT

Elaborado por: José Andrés Ureta Granda
Fuente: Elaboración propia a partir de [1], [7]

El costo de la atención añade un término que depende de la longitud de contexto. En el prefill, cada token del prompt atiende a los anteriores, de modo que el cálculo de la atención crece con el cuadrado de L_{in}; con prompts largos, este término gana peso frente al de las capas lineales. FlashAttention reduce los accesos a memoria de este cálculo procesándolo por bloques, sin alterar su resultado [9]. En el decode, cada paso lee las claves y los valores de todo el contexto acumulado, por lo que el volumen leído por paso crece con la longitud de la secuencia y con el número de solicitudes del lote.

1.7.3 KV cache

El KV cache es la estructura en la que el motor de inferencia conserva, en la memoria de la GPU, las claves y los valores de cada token ya procesado. Lo crea la fase prefill y lo amplía cada paso de la fase decode. Su tamaño por token se obtiene con la Ecuación 2, donde n_{capas} es el número de capas, n_{KV} el número de cabezas de clave y valor, d_h la dimensión de cada cabeza y b el número de bytes por elemento. El factor 2 corresponde a las claves y los valores.

$$m_{KV} = 2 \times n_{capas} \times n_{KV} \times d_h \times b \tag{2}$$

La memoria total que ocupa el KV cache crece de forma lineal con el número de tokens almacenados, como recoge la Ecuación 3. Para B solicitudes con longitudes de secuencia $L_1, \ldots, L_B$, la suma depende a la vez de la longitud de contexto y de la concurrencia, las dos variables que este trabajo hace variar.

$$M_{KV} = m_{KV} \sum_{i=1}^{B} L_i \tag{3}$$

La memoria de la GPU se reparte entre los pesos del modelo, las activaciones y el KV cache. Una vez cargados los pesos, la memoria restante fija cuántos tokens pueden almacenarse y, con ello, cuántas solicitudes pueden atenderse de forma simultánea. El valor de m_{KV} para los dos modelos y el presupuesto de memoria de las configuraciones más exigentes se calculan en la Ecuación 15 y siguientes del Capítulo II.

1.7.4 Gestión de memoria y planificación en vLLM

vLLM gestiona el KV cache mediante PagedAttention [4]. Este mecanismo divide el KV cache de cada solicitud en bloques de un número fijo de tokens, que no necesitan ocupar posiciones contiguas en la memoria. Una tabla de bloques asocia las posiciones lógicas de cada secuencia con los bloques físicos que las almacenan, de forma análoga a la tabla de páginas de la memoria virtual de un sistema operativo. Los bloques se asignan a medida que la secuencia crece, de modo que el desperdicio queda limitado al último bloque, que puede estar parcialmente ocupado. La misma indirección permite que varias solicitudes compartan los bloques de un prefijo común [4].

Cuando el KV cache se llena, vLLM recurre a la expulsión (preemption) de solicitudes: libera los bloques de una o más de ellas y, cuando vuelve a haber memoria, las reanuda trasladando sus bloques a la memoria de la CPU o recalculándolos [4]. La expulsión mantiene el servicio en funcionamiento, pero alarga la latencia de las solicitudes afectadas. Por ello, el número de expulsiones es un indicador de que la configuración opera en el límite de la memoria.

La planificación de solicitudes sigue el esquema de batching continuo introducido por Orca [8]. El planificador decide el contenido del lote en cada iteración del modelo y no una vez por solicitud: incorpora solicitudes nuevas en cuanto hay capacidad y retira las finalizadas sin esperar al resto del lote. En consecuencia, un mismo lote puede contener solicitudes en prefill y solicitudes en decode.

Esa convivencia genera interferencia entre fases. Un prefill largo ocupa la GPU

durante una iteración costosa y retrasa el paso siguiente de todas las solicitudes en decode, lo que eleva su TPOT [7]. El chunked prefill atenúa este efecto: divide el prefill en fragmentos y limita el número de tokens procesados en cada iteración, de modo que el decode avanza sin detenerse mientras se completa el prefill [7]. La contrapartida es un TTFT mayor para la solicitud cuyo prefill se fragmenta.

El comportamiento de estos mecanismos se controla mediante parámetros de configuración del motor. Los principales son la fracción de la memoria de la GPU que vLLM puede reservar (`gpu_memory_utilization`), el número máximo de solicitudes simultáneas en un lote (`max_num_seqs`) y el número máximo de tokens por iteración (`max_num_batched_tokens`), que acota el tamaño de los fragmentos del chunked prefill. vLLM combina, además, la ejecución del modelo mediante grafos CUDA con la ejecución inmediata (*eager*) de PyTorch; la opción `enforce_eager` desactiva los grafos y ejecuta siempre en modo inmediato.[1] La configuración base de estos parámetros se fija en el Capítulo II.

1.7.5 Representación numérica y cuantización

La representación numérica determina cuántos bytes ocupa cada peso y, con ello, el volumen que se lee de memoria en cada pasada. BF16 es un formato de coma flotante de 16 bits que conserva el rango dinámico de la coma flotante de 32 bits y reduce la precisión de la mantisa [21]. La A100 ejecuta operaciones BF16 en sus Tensor Cores [3]. En BF16, cada peso ocupa 2 bytes.

La cuantización representa los pesos con menos bits. AWQ es un método de cuantización de solo pesos: los pesos se almacenan en enteros de 4 bits (INT4), mientras que las activaciones y el cálculo se mantienen en coma flotante de 16 bits [10]. El método parte de la observación de que una pequeña fracción de los pesos es determinante para la calidad del modelo, y que esa fracción puede identificarse a partir de la magnitud de las activaciones y no de los propios pesos. Para protegerla, AWQ escala los canales correspondientes antes de cuantizar, de modo que su error de cuantización relativo disminuye [10].

El efecto esperable de la cuantización difiere entre fases. En el decode, limitado por el ancho de banda de memoria, leer 4 bits por peso en lugar de 16 reduce el volumen leído por paso, que es el recurso limitante [10]. En el prefill, limitado por el cómputo, esa reducción no alivia el recurso limitante, y los pesos deben convertirse a coma flotante antes de operar con ellos. La memoria que libera la cuantización

[1] Definiciones tomadas de la documentación de los parámetros en el código fuente de vLLM 0.20.0 (`vllm/config/cache.py`, `scheduler.py` y `model.py`), disponible en `https://github.com/vllm-project/vllm/tree/v0.20.0/vllm/config`. Consultado el 24 de octubre de 2026.

queda, además, disponible para el KV cache y, con ello, para atender más solicitudes. La reducción de bits puede degradar la calidad de las respuestas, por lo que el alcance incluye la comparación limitada de calidad descrita en la Sección 1.6.1. Que estos efectos se produzcan, y en qué magnitud, para Qwen3-14B en la A100 es una de las preguntas que el trabajo responde experimentalmente.

1.7.6 Modelo roofline, MFU y MBU

El modelo roofline acota el rendimiento alcanzable por un cálculo a partir de dos límites del procesador: su rendimiento máximo de cómputo y el ancho de banda de su memoria [19]. El modelo caracteriza cada cálculo por su intensidad aritmética I, el cociente entre las operaciones de coma flotante que realiza y los bytes que transfiere desde la memoria. El rendimiento alcanzable P es el menor de los dos límites, como expresa la Ecuación 4, donde P_{pico} es el rendimiento máximo de cómputo y BW el ancho de banda de memoria.

$$P = \text{mín}\left(P_{pico},\ I \times BW\right) \tag{4}$$

Los dos límites se cruzan en el punto de equilibrio $I^* = P_{pico}/BW$. Un cálculo con $I < I^*$ queda limitado por la memoria, y uno con $I > I^*$, por el cómputo. Para la A100 de 40 GB, con 312 TFLOP/s en BF16 denso y 1555 GB/s de ancho de banda [3], el punto de equilibrio se sitúa en torno a 200 operaciones por byte.

Este marco traduce a cifras la diferencia descrita en la Sección 1.7.2. Como la intensidad del prefill crece con L_{in}, un prompt de varios cientos de tokens supera el punto de equilibrio y queda limitado por el cómputo. Como la intensidad del decode crece con B, un lote de pocas solicitudes queda muy por debajo del punto de equilibrio y el paso queda limitado por la lectura de los pesos y del KV cache. El modelo proporciona así una cota superior del rendimiento de cada fase, cuyo cálculo para Qwen3-14B y Qwen3-8B se realiza en la Sección 2.7.2.

Dos métricas expresan qué fracción de esa cota se alcanza. La utilización de FLOPs del modelo (MFU, por sus siglas en inglés) es el cociente entre el throughput observado y el máximo teórico que alcanzaría el sistema operando a P_{pico}, contando solo las operaciones que el modelo exige [22]; resulta adecuada para el prefill, limitado por el cómputo. La utilización del ancho de banda del modelo (MBU) es el cociente entre los bytes por segundo que el modelo lee efectivamente, pesos y KV cache incluidos, y BW; resulta adecuada para el decode, limitado por la memoria [23]. Un valor bajo de la métrica correspondiente indica que la fase no

aprovecha el recurso que la limita, y orienta la búsqueda del cuello de botella hacia otras causas, como la planificación o la interferencia entre fases.

1.7.7 Métricas de desempeño de la inferencia

La caracterización por fases exige métricas que separen la contribución de cada una a la latencia percibida. El TTFT es el tiempo que transcurre desde el envío de la solicitud hasta la recepción del primer token; refleja la fase prefill y, en un servidor con carga, también el tiempo de espera en cola. El TPOT es el tiempo medio entre tokens de salida consecutivos de una solicitud y refleja la fase decode [1]. La latencia extremo a extremo (E2E) es el tiempo total desde el envío hasta el último token. Las tres se relacionan mediante la Ecuación 5, que define el TPOT a partir de las otras dos.

$$\mathrm{TPOT} = \frac{\mathrm{E2E} - \mathrm{TTFT}}{L_{out} - 1} \tag{5}$$

El TPOT es un promedio por solicitud. La latencia entre tokens (ITL, por sus siglas en inglés) registra, en cambio, cada intervalo individual entre dos tokens consecutivos. Su distribución revela pausas que el promedio oculta, como las que produce un prefill largo al interferir con el decode [7].

El throughput mide la capacidad del sistema y se expresa en tokens de salida por segundo o en solicitudes atendidas por segundo. El goodput restringe ese recuento a las solicitudes que cumplen unos objetivos de nivel de servicio (SLO) sobre el TTFT y el TPOT [1]. A diferencia del throughput, el goodput penaliza una configuración que procesa muchos tokens a costa de una latencia inaceptable para el usuario.

La energía se estima a partir de la potencia que informa la telemetría interna de la GPU, mediante NVML y DCGM. La energía consumida durante un intervalo es la integral de la potencia en ese intervalo, y la energía por token se obtiene con la Ecuación 6, donde $p(t)$ es la potencia instantánea entre los instantes t_0 y t_1, y N_{tok} el número de tokens generados en ese intervalo.

$$E_{tok} = \frac{1}{N_{tok}} \int_{t_0}^{t_1} p(t)\, dt \tag{6}$$

Como precisa la Sección 1.6.1, esta energía es una estimación basada en la telemetría y no una medición eléctrica calibrada. Con estas métricas, los mecanismos descritos en esta sección y las cotas del modelo roofline, el Capítulo II establece el método con que se miden las dos fases.

CAPÍTULO II

METODOLOGÍA

2.1 Diagnóstico de la Situación Actual

El método de este trabajo no se diseñó en abstracto, sino sobre una infraestructura concreta cuyas características imponen condiciones a la medición. Por esa razón, el capítulo parte de un diagnóstico de la plataforma disponible antes de describir el método. Las limitaciones identificadas aquí reaparecen en las secciones siguientes como el porqué de buena parte de las decisiones metodológicas, y varias de ellas no se conocían al comenzar: se descubrieron al intentar medir, como describe la Sección 2.3.

La plataforma experimental es el clúster de computación de alto rendimiento de CEDIA (`hpc.cedia.edu.ec`), gestionado con el planificador de trabajos Slurm [24]. Se eligió esta plataforma porque es la que ofrece acceso a la GPU NVIDIA A100 que fija el caso de estudio, y no por preferencia frente a otras infraestructuras. La Tabla 3 resume sus características. Los datos de rendimiento de la GPU proceden de la hoja de especificaciones del fabricante [3].

Tabla 3: Características de la plataforma experimental.

Elemento	**Valor**
Clúster	HPC CEDIA, planificador Slurm
Nodos de cómputo GPU	`compute-0-1` y `compute-0-2`, con 8 NVIDIA A100-SXM4-40GB cada uno
Nodo interactivo	`compute-0-0`, con 1 A100-40GB, 6 particiones MIG `a100_2g.10gb` y 2 particiones MIG `a100_1g.5gb`
CPU por nodo	256 núcleos
RAM por nodo	≈960 GB (nodos GPU); ≈1 TB (nodo interactivo)
Memoria de la GPU	40 960 MiB
Límite de potencia de la GPU	400 W
Ancho de banda HBM2	1555 GB/s
Rendimiento BF16 con Tensor Cores	312 TFLOPS (denso)
Sistema operativo	Ubuntu 20.04.6, glibc 2.31
Almacenamiento de trabajo	NFS compartido (`$HOME`); disco local por trabajo en `/scratch`

Elaborado por: José Andrés Ureta Granda

Fuente: Inspección de los nodos del clúster HPC CEDIA; hoja de especificaciones de la A100 [3]

La variante de la GPU es la A100-SXM4 de 40 GB, dato que la ficha institucional exige documentar porque la capacidad de memoria condiciona los niveles máximos de longitud de contexto y concurrencia. Esa capacidad pesa más con el modelo principal del trabajo, Qwen3-14B, cuyos pesos en BF16 ocupan 27,52 GiB según el registro del propio motor, es decir, cerca de dos tercios de la GPU. El sistema operativo de los nodos es Ubuntu 20.04.6, mientras que la ficha sugiere Ubuntu 22.04 o equivalente. La diferencia no es una elección del trabajo, sino una condición del clúster que no puede modificarse. Su consecuencia más importante es la versión de glibc, la 2.31, que origina el problema de compatibilidad tratado en la Sección 2.6.3.

El nodo interactivo ofrece particiones MIG (*Multi-Instance GPU*), que dividen una A100 en instancias aisladas de menor capacidad. Estas particiones quedan excluidas de la campaña de medición por dos razones. La primera es que su memoria, de 5 o 10 GB, resulta insuficiente para alojar Qwen3-8B y, con mayor razón, Qwen3-14B; solo sirvieron para pruebas de humo con modelos pequeños. La segunda es que NVML no expone la potencia por instancia MIG, lo que impediría atribuir el consumo energético que exige el tercer objetivo específico.[1]

La exclusión de MIG obliga, además, a una precaución de identificación. En un trabajo asignado a una partición MIG, `nvidia-smi` muestra la A100 de 40 GB que la contiene y no la instancia efectiva. Por ello, el dispositivo real se identifica con `nvidia-smi -L`, `torch.cuda.get_device_name()` o `torch.cuda.mem_get_info()`, y no con la salida general de `nvidia-smi`.

El planificador impone los límites recogidos en la Tabla 4. A ellos se suman dos restricciones de la calidad de servicio (QoS) `normal` asignada al usuario: solo se admite un trabajo en ejecución simultánea y no es posible reservar un nodo completo. La partición por defecto es `interactive`, que solo ofrece particiones MIG. En consecuencia, omitir la partición en la solicitud conduce a un recurso inservible para este experimento.

Elaborado por: José Andrés Ureta Granda

Fuente: Configuración de Slurm del clúster HPC CEDIA

Tres limitaciones adicionales afectan directamente a la validez y al costo de

[1] La documentación de NVML define `nvmlDeviceGetPowerUsage` y `nvmlDeviceGetTotalEnergyConsumption` sobre la GPU completa y sus circuitos asociados, sin declarar soporte para los manejadores de instancia MIG; en cambio, consultas como la de memoria sí indican que la información por instancia se obtiene con esos manejadores. NVIDIA, *NVML API Reference Guide*, disponible en `https://docs.nvidia.com/deploy/nvml-api/`. Consultado el 24 de octubre de 2026.

Tabla 4: Límites de las particiones de Slurm disponibles para el usuario.

Partición	Tiempo máx.	GPU máx.	CPU máx.	RAM máx.
`interactive` (por defecto)	1 día	—	—	—
`gpu`	2 días	2	160	128 G
`gpu-max`	1 día	4	128	256 G
`gpu-dev`	4 días	1	16	—

las mediciones. La primera es que los nodos son compartidos con otros usuarios y su carga de CPU varía durante la ejecución, como se detalla en la Sección 2.4.2. La segunda es que los nodos GPU no comparten la misma versión del driver, según muestra la Tabla 5. Como el planificador asigna el nodo en cada trabajo, dos mediciones de la misma configuración pueden ejecutarse con drivers distintos y no ser directamente comparables. La tercera es que el entorno de software y los pesos del modelo residen en el NFS compartido, y su lectura desde los nodos resultó ser el costo dominante del arranque del servidor con Qwen3-14B, como cuantifica la Sección 2.3.

Tabla 5: Versión del driver NVIDIA observada en los nodos.

Fecha	Nodo	Driver
20 de octubre	`compute-0-1`	535.161.08
22 de octubre	`compute-0-2`	575.51.03
24 de octubre	`compute-0-1` (trabajo 30732)	535.161.08
27 de octubre	`compute-0-1` (trabajo 31157)	535.161.08

Elaborado por: José Andrés Ureta Granda
Fuente: Registros `entorno.txt` de los trabajos ejecutados

El diagnóstico deja establecidas, por tanto, las condiciones de partida: una A100 de 40 GB sin reserva exclusiva del nodo, un sistema operativo con glibc 2.31, un driver que puede cambiar entre ejecuciones, un almacenamiento compartido lento para archivos voluminosos y un único trabajo simultáneo. Antes de describir cómo se trabajó sobre esas condiciones, la Sección 2.2 delimita el área de estudio.

2.2 Área de Estudio

La Tabla 6 sitúa el trabajo en su dominio académico y precisa el objeto de estudio. Su contenido se deriva de la ficha institucional aprobada, con el cambio de modelo principal acordado con el tutor: Qwen3-14B pasa a ser el objeto de la campaña, y Qwen3-8B se conserva como piloto.

Tabla 6: Área de estudio.

Dominio:	Sistemas de inferencia de modelos de lenguaje de gran tamaño sobre infraestructura de computación de alto rendimiento
Línea de investigación:	Diseño, desarrollo e integración de sistemas basados en arquitecturas web, móviles y sistemas embebidos
Sublínea de investigación:	Optimización del rendimiento.
Campo:	Tecnologías de la información y la comunicación (TIC)
Área:	Caracterización y modelado del rendimiento de la inferencia de LLM en GPU
Aspectos:	Comportamiento de las fases prefill y decode; TTFT, TPOT, throughput, memoria, utilización y energía estimada; efecto de la longitud de contexto y la concurrencia
Objeto de estudio:	Modelo Qwen3-14B en BF16 servido con vLLM sobre una GPU NVIDIA A100-SXM4-40GB del clúster HPC CEDIA, con Qwen3-8B como piloto de calibración
Periodo de estudio:	Desde el 21 de octubre de 2026 (instalación del entorno) hasta el cierre de la campaña con Qwen3-14B

El cambio de modelo responde a una indicación del tutor y tiene una razón técnica. Con Qwen3-14B los pesos ocupan la mayor parte de la memoria de la GPU, y el espacio restante para el KV cache se reduce a una fracción del que deja Qwen3-8B. El modelo mayor sitúa así la A100 de 40 GB cerca de su límite de memoria, que es precisamente la condición en la que la separación entre prefill y decode tiene más consecuencias prácticas. La Sección 2.7.2 cuantifica ese margen. Con el objeto de estudio delimitado, la Sección 2.3 describe el método con que se caracteriza.

2.3 Metodología de Desarrollo o Implementación

El trabajo sigue una metodología de prueba y error, de carácter iterativo, sobre la que se apoya la medición cuantitativa. Se adoptó porque el entorno no podía especificarse por completo de antemano: ni la documentación del clúster ni la de las herramientas anticipaban que la rueda de vLLM fallaría con glibc 2.31, que el arranque del servidor tardaría casi una hora desde el NFS o que un servidor alternativo calcularía mal la memoria libre. Cada uno de esos hechos se descubrió al intentar medir. Un plan cerrado habría fallado en el primero de ellos; un proceso iterativo convierte cada fallo en información para el intento siguiente.

Cada iteración consta de cuatro pasos. El primero es el **intento**: se ejecuta una configuración concreta, definida por el motor, el modelo, la partición y la ubicación del entorno y los pesos. El segundo es la **observación**: se registra qué se midió o en qué punto falló, siempre asociado al identificador del trabajo de Slurm que lo produjo. El tercero es el **diagnóstico**: se busca la causa en los registros y, cuando no se encuentra, se declara como no diagnosticada en lugar de suponerla. El cuarto es el **ajuste**: se modifica una sola cosa para el intento siguiente, de modo que el efecto del cambio pueda atribuirse a esa modificación.

La prueba y error se aplica a la construcción del entorno y del protocolo, no a los datos. Una vez que una configuración funciona, la medición sigue un protocolo fijo con repeticiones, calentamiento descartado y registro de metadatos, descrito en la Sección 2.4. Ninguna medición se descarta por su valor; solo se invalida cuando el registro muestra una causa externa al fenómeno medido, como la descarga parcial del modelo a la CPU, y esa invalidación se declara junto con su motivo.

La Tabla 7 recoge las iteraciones realizadas hasta la fecha. Su lectura en orden muestra cómo el método fue cambiando: de un barrido amplio con reinicio del motor por configuración a un servidor único con un cliente propio, de Qwen3-8B a Qwen3-14B, y del entorno en el NFS a su copia en el disco local del nodo.
Elaborado por: José Andrés Ureta Granda

Tabla 7: Iteraciones de prueba y error del trabajo.

N.º	Fecha y trabajo	Intento	Observación y diagnóstico	Ajuste
1	21–24 oct	Instalar vLLM 0.20.0 en un entorno Python	`ImportError`: la rueda pide `GLIBC_2.34` y el nodo tiene 2.31; etiqueta de la rueda incorrecta	Parche `parche_glibc.sh` (Sección 2.6.3); prueba de humo superada
2	24 oct, 30732	Barrido piloto de Qwen3-8B con `bench_pd.sh`	Límite de 5 h alcanzado; la entrada de 128 tokens sola tardó ≈2,5 h. Medidas válidas solo con lote 1	Reutilizar un único servidor y medir con un cliente propio
3	25–27 oct	Cliente `tfgbench` con Qwen3-0.6B en MIG de 5 GB	Piloto completo; el instrumento queda validado	Lanzar la campaña de Qwen3-8B con el cliente
4	27 oct, 31157	Campaña de Qwen3-8B	El entorno tardó 21 min en prepararse y `vllm serve` no escribió nada en 45 min; sin diagnóstico	Registrar los tiempos del arranque
5	29 oct, 31299	Cambio a Qwen3-14B por indicación del tutor	Sesión interactiva colgada: `spack load hwloc` en `.bashrc`	Corregido el `.bashrc`
6	2 nov	Línea base del arranque de Qwen3-14B desde el NFS	Terminó a los 59 min 31 s por el límite interno de vLLM, 4 min 20 s después de cargar los pesos	Copiar el entorno y los pesos a `/scratch`
7	8 y 11 nov, 32710 y 33107	Copia del entorno a `/scratch`	Pesos copiados en 459 s y 414 s; del entorno virtual (12 GB) solo 1,2 GB en 71 min, con load average ≈790	Pendiente: reducir el costo de la copia del entorno
8	11 nov, 33107	Contraste con Ollama 0.24.0	`qwen3-coder:30b` medido; `nemotron-3-nano:30b` cargado en parte en CPU por una lectura desactualizada de la memoria libre	Reiniciar el servidor entre modelos

Fuente: Registros de los trabajos de Slurm y `RESUMEN_TESIS.md`

La iteración 2 fijó la decisión metodológica más importante: reutilizar el motor de inferencia. Arrancar vLLM tiene un costo fijo elevado: en el trabajo 30732, la inicialización del motor con Qwen3-8B tardó entre 4 y 24 minutos, dominada por la compilación con `torch.compile`, y cada ejecución completa costó entre 29 y 75 minutos, como detalla la Sección 3.2.1. Reiniciar el motor para cada configuración multiplicaría ese costo por el número de configuraciones dentro de un único trabajo con tiempo limitado. Por ello, a partir de la iteración 3 el servidor se levanta una sola vez y un cliente propio recorre todas las configuraciones contra él.

La iteración 6 mostró que, con Qwen3-14B, el arranque es en sí mismo un problema de medición. El registro del servidor permite descomponer los 59 min 31 s del intento. Pasaron 19 min 29 s entre el lanzamiento y la primera línea de registro de vLLM, tiempo que corresponde a la importación de la pila de Python desde el NFS. El proceso del motor no se inició hasta 26 minutos después. La lectura de los 27,51 GiB de pesos, también desde el NFS, se completó en 110 s, y el modelo quedó cargado en la GPU a los 285,9 s de empezar esa fase. Cuatro minutos y veinte segundos después, el servidor abandonó el arranque con un `TimeoutError`, porque había superado los 600 s que concede por defecto la variable `VLLM_ENGINE_READY_TIMEOUT_S` para que el motor esté listo. El diagnóstico es que el cuello de botella no está en la GPU, sino en la lectura de miles de archivos pequeños desde el almacenamiento compartido.

De ese diagnóstico se derivan las dos líneas de ajuste en curso. La primera, ensayada en la iteración 7, es copiar el entorno virtual y la caché de modelos al disco local del nodo antes de arrancar. La copia de los pesos funcionó en unos 7 minutos, pero la del entorno virtual no avanzó a un ritmo útil con el nodo sobrecargado, lo que confirma que el costo está en el número de archivos y no en su volumen. La segunda es elevar el límite de arranque de vLLM y cronometrar por separado cada fase del arranque con el script `fase2_arranque.sh`, que espera hasta 15 minutos a que el servidor responda y no lo detiene si se excede, para que la decisión de continuar quede en manos del investigador.

La iteración 8 introdujo un segundo servidor de inferencia, Ollama, como contraste. Ollama sirve modelos cuantizados en formato GGUF y expone en cada respuesta la duración de su prefill y de su decode, lo que ofrece una perspectiva independiente de las dos fases. Se usó la versión 0.24.0 porque las versiones a partir de la 0.30 exigen un driver 550 o superior, por encima del 535 del nodo. Sus resultados no sustituyen a la campaña con vLLM: comparten la GPU y el protocolo, pero no el modelo ni la representación numérica, de modo que solo se usan para contrastar tendencias. La iteración dejó, además, una regla operativa: entre dos modelos, el ser-

vidor debe reiniciarse, porque con un dato de memoria libre desactualizado repartió el segundo modelo entre la GPU y la CPU, y esa medición tuvo que invalidarse.

Las restricciones de Slurm, por su parte, determinaron dos decisiones que se mantienen en todas las iteraciones. La primera es la ejecución secuencial de las configuraciones: con un único trabajo simultáneo, no es posible paralelizarlas entre GPU. La segunda es la elección de la partición `gpu-dev` en lugar de `gpu`. El 24 de octubre, `sbatch --test-only` estimaba en `gpu` un inicio para el 31 de octubre pese a haber GPU libres, mientras que en `gpu-dev` la estimación era inmediata; `gpu-dev` ofrece, además, 4 días de tiempo máximo de ejecución frente a 2.

Cada trabajo de medición se organiza en las fases de la Tabla 8. Se automatizó todo el recorrido porque cada trabajo de Slurm debe ser autosuficiente: el investigador no interviene entre fases, y cualquier paso manual introduciría variaciones no registradas. Cada fase deja sus salidas en un directorio propio del trabajo, de modo que toda medición queda asociada al identificador del trabajo que la produjo. Las fases son las mismas que estableció `bench_pd.sh` en la iteración 2, con dos cambios aprendidos después: la preparación del entorno en el disco local y el cronometraje del arranque como fase propia.

Tabla 8: Fases de un trabajo de medición.

Fase	Contenido	Salidas
0	Registro del nodo, la GPU, las versiones y la carga; arranque del monitor de telemetría	`entorno.txt`, `gpu_monitor.csv`
1	Copia del entorno y de los pesos a `/scratch` y verificación de la importación de vLLM	Registro de la fase 1
2	Arranque del servidor, cronometrado hasta que `/health` responde	`serve.log`, tiempo de arranque
3	Barrido en línea con el cliente contra el servidor único	Un archivo JSON por configuración; `/metrics` del servidor
4	Agregación y tablas	Resumen en JSON y Markdown

Elaborado por: José Andrés Ureta Granda
Fuente: Elaboración propia a partir de `bench_pd.sh`, `fase1_preparar.sh` y `fase2_arranque.sh`

El orden de las fases responde a un criterio de economía del tiempo de GPU. La fase 0 registra el entorno antes de medir, porque el driver y la carga del nodo pueden cambiar entre trabajos. Las fases 1 y 2 verifican que el entorno y el servidor funcionan antes de comprometer el resto del tiempo asignado, de modo que un fallo se detecta antes de invertir horas de GPU en un barrido inválido. Esa separación es también una lección de las iteraciones 4 y 6, en las que el tiempo se perdió en el

arranque sin que quedara registro de en qué parte.

La matriz experimental tampoco quedó fijada de antemano: se calibra con los pilotos, tal como prevé la ficha en la mitigación de su riesgo R6, que propone ejecutar un piloto, cronometrar cada configuración y renegociar la matriz con el tutor a partir de ese dato. La Tabla 9 compara la matriz de la ficha con el piloto ejecutado con Qwen3-8B y con los barridos preparados para Qwen3-14B: el experimento E1, dedicado al prefill, y el experimento E2, dedicado al decode por lotes.

Tabla 9: Matriz de la ficha, piloto con Qwen3-8B y barridos preparados para Qwen3-14B.

Variable	**Ficha aprobada**	**Piloto Qwen3-8B (30732)**	**Qwen3-14B (E1 y E2)**
Variante numérica	BF16 y AWQ INT4	BF16	BF16
Longitud de entrada	128, 512, 2048 y 8192	128, 512, 1024, 2048 y 4096	E1: 128 a 7680 en 7 puntos; E2: 1024
Longitud de salida	64 y 256	1 y 129 (sin servidor)	E1: 128; E2: 256
Concurrencia	1, 4, 8 y 16	1 (medidas válidas)	E1: 1; E2: 1 a 64 en 7 puntos
Repeticiones	5	10 iteraciones	E1: 30 + 3 de calentamiento; E2: 256 solicitudes + 4

Elaborado por: José Andrés Ureta Granda
Fuente: Ficha institucional ITI-TFG-B26-0017; `bench_pd.sh`; `e1_prefill.toml` y `e2_concurrencia.toml`

Tres divergencias siguen abiertas y la memoria las declara en lugar de omitirlas. La primera es la variante AWQ INT4 [10], que constituye la mitad de la comparación prevista en la ficha y que ninguna iteración ha abordado todavía. La segunda es la concurrencia máxima de E2: con Qwen3-14B, el cálculo de la Sección 2.7.2 indica que 32 y 64 secuencias de 1280 tokens no caben en el KV cache disponible, de modo que esos puntos medirán la espera o la expulsión de solicitudes además del decode. La tercera es el chunked prefill, que puede mezclar ambas fases en un mismo lote [7], según advierte el riesgo R4 de la ficha; la decisión de desactivarlo o documentarlo sigue pendiente. La ficha prevé, además, aleatorizar el orden de las configuraciones, y la campaña con Qwen3-14B deberá incorporarlo. Con la organización de la medición descrita, la Sección 2.4 precisa qué se mide y con qué instrumentos.

2.4 Técnicas e Instrumentos de Recolección de Información

La técnica central del trabajo es la medición instrumentada de la inferencia. Las métricas por fase son el TTFT (*time to first token*), que caracteriza el prefill, y el TPOT (*time per output token*), que caracteriza el decode. Ninguna de las dos se observa directamente en un único instrumento sin supuestos, y por esa razón se triangulan con tres métodos independientes. Si los tres métodos convergen, la medición es robusta; si divergen, la magnitud de la discrepancia acota el error de cada uno y constituye un hallazgo que merece discusión.

2.4.1 Triangulación de la separación prefill/decode

El primer método obtiene las fases por diferencia de latencias en el modo sin servidor, con `vllm bench latency`. Con una longitud de salida de un token, el tiempo medido corresponde esencialmente al prefill; con 129 tokens, se añaden 128 pasos de decode. Se eligió la salida de 129 tokens para que la diferencia contenga exactamente 128 pasos de decode. Las Ecuaciones 7 a 9 formalizan el procedimiento, donde $T(n)$ es la latencia medida con n tokens de salida, L_{in} la longitud de entrada y B el tamaño de lote. Este método se aplicó en el piloto con Qwen3-8B; con Qwen3-14B, el costo del arranque hace preferible medir siempre contra un servidor ya levantado.

$$T_{prefill} \approx T(1) \tag{7}$$

$$X_{prefill} = \frac{L_{in} \times B}{T(1)} \tag{8}$$

$$\text{TPOT} = \frac{T(129) - T(1)}{128} \tag{9}$$

La ventaja de este método es que aísla las fases sin instrumentación interna del motor. Su limitación es que $T(1)$ no es prefill puro: incluye una pequeña fracción de decode y el sobrecoste de arranque de la iteración. Por ello, la Ecuación 7 es una aproximación y su error debe contrastarse con los otros dos métodos.

El segundo método recoge las métricas de cliente en el modo en línea, contra un servidor `vllm serve`. El cliente propio `tfgbench`, cuya salida es compatible con la de `vllm bench serve`, envía cada solicitud en streaming y marca el instante en

que llega cada fragmento de texto; de esas marcas obtiene el TTFT, el TPOT y el ITL (*inter-token latency*, latencia entre tokens consecutivos). Cada solicitud genera exactamente la longitud de salida pedida, con la opción `ignore_eos`, aunque el modelo emita antes el token de fin de secuencia; sin ella, solicitudes de la misma configuración tendrían longitudes distintas y su TPOT no sería comparable. Este método ofrece la perspectiva del usuario, y su limitación es que el TTFT incluye el tiempo de espera en cola del servidor, además del prefill.

El tercer método lee las métricas internas del servidor. En vLLM se obtienen del endpoint `/metrics`, cuyos histogramas `request_prefill_time_seconds` y `request_decode_time_seconds` separan las fases según la contabilidad interna del motor. En Ollama, el equivalente son los campos `prompt_eval_duration` y `eval_duration` que acompañan a la última respuesta de cada solicitud. Es la medida más directa y por eso sirve de referencia para validar los dos métodos anteriores. Su limitación es que depende de cómo cada motor define el inicio y el fin de cada fase, definición que no es observable desde fuera y que puede variar entre versiones.[2]

La triangulación ya produjo un hallazgo en la iteración 8. Con Ollama, el TTFT medido en el cliente superó al prefill reportado por el servidor en 1,0 a 2,4 s por solicitud, una diferencia que coincide con el campo `load_duration` que el servidor reporta en cada solicitud aunque el modelo ya esté cargado en la GPU. La red y el cliente aportan alrededor de 1 ms, de modo que la diferencia se origina en el servidor; su causa no está diagnosticada. Sin el contraste entre el método 2 y el método 3, ese costo habría quedado atribuido al prefill.

2.4.2 Telemetría, energía y control de la contención

La telemetría de la GPU se registra con un monitor de `nvidia-smi` que muestrea durante toda la ejecución y escribe `gpu_monitor.csv`. Se recurrió a `nvidia-smi` porque está disponible en los nodos y expone la memoria, la utilización, la temperatura y la potencia que exige el tercer objetivo específico. La ficha prevé DCGM para la campaña principal y Nsight Systems solo en un subconjunto representativo, por el volumen de datos y el sobrecoste del perfilado.

La energía se calculará a partir de esa telemetría con las Ecuaciones 10 y 11, definidas en la ficha. En ellas, P_i es la potencia de la muestra i, Δt el periodo de

[2]Las opciones de `vllm bench serve` (`--ignore-eos`, `--metric-percentiles`) y los histogramas `vllm:request_prefill_time_seconds` y `vllm:request_decode_time_seconds` se documentan en el código fuente de vLLM 0.20.0 (`vllm/benchmarks/serve.py` y `vllm/v1/metrics/loggers.py`), disponible en `https://github.com/vllm-project/vllm/tree/v0.20.0`. Consultado el 24 de octubre de 2026. Los campos de duración de Ollama se describen en la documentación de su API, disponible en `https://github.com/ollama/ollama/blob/main/docs/api.md`. Consultado el 11 de noviembre de 2026.

muestreo, E_{idle} la energía consumida en reposo durante el mismo intervalo y N_{tokens} el número de tokens generados. La energía así obtenida es una estimación basada en la telemetría interna de la GPU y no una medición eléctrica calibrada externamente. La ventana de integración y la medición de E_{idle} en cada campaña quedan por fijar en el protocolo de energía.

$$E = \sum_{i=1}^{n} P_i \, \Delta t \tag{10}$$

$$E_{token} = \frac{E_{ejecucion} - E_{idle}}{N_{tokens}} \tag{11}$$

La contención del nodo es una variable de confusión, porque el TTFT y el TPOT dependen de que la CPU alimente a la GPU sin demora. Su magnitud quedó documentada desde la preparación: el 22 de octubre a las 19:34, `compute-0-2` registraba un load average de 317,33, 337,72 y 214,73 en las ventanas de 1, 5 y 15 minutos, sobre 256 núcleos, es decir, una sobresuscripción de 1,24 veces en la ventana de un minuto. En noviembre la situación empeoró: el 11 de noviembre, `compute-0-1` pasó de un load average de 260–380 durante la medición con Ollama a cerca de 790 una hora después, más de tres veces el número de núcleos.

Como el usuario no puede reservar un nodo completo, la contención no puede eliminarse y se controla de tres formas. La primera es registrar `/proc/loadavg` al inicio y al final de cada trabajo, para que la carga quede asociada a cada medición. La segunda es el monitor de `nvidia-smi` descrito, que permite detectar intervalos de baja utilización atribuibles a la CPU. La tercera es ejecutar repeticiones suficientes para estimar la varianza y detectar ejecuciones contaminadas, en lugar de confiar en una medición única.

La diferencia de driver entre nodos de la Tabla 5 se controla con el mismo principio de registro. Cada ejecución guarda el nodo y la versión del driver vigente en el momento de medir. La campaña definitiva deberá ejecutarse en un mismo nodo o, si el planificador no lo permite, reportar el driver como covariable en el análisis. Definidos los instrumentos y sus controles, la Sección 2.5 fija los requerimientos que debe cumplir el banco de pruebas que los integra.

2.5 Requerimientos del Proyecto

Los requerimientos del banco de pruebas se derivan de los objetivos específicos y de las condiciones identificadas en el diagnóstico. Se formularon como requerimientos verificables porque el banco es un instrumento de medición, y un instrumento sin especificación no puede validarse. Varios de ellos no figuraban en la especificación inicial: surgieron de las iteraciones de la Tabla 7, y la columna de origen lo indica. Las Tablas 10 y 11 los recogen junto con la razón de cada uno.

2.5.1 Requerimientos Funcionales

Tabla 10: Requerimientos funcionales del banco de pruebas.

ID	Requerimiento	Razón	Origen
RF1	Levantar el servidor de inferencia dentro del trabajo de Slurm con una configuración versionada y no enviar carga hasta que responda su comprobación de salud (`/health` en vLLM, `/api/version` en Ollama)	Evita medir contra un servidor que aún no está listo	SPEC; it. 6
RF2	Cronometrar el arranque del servidor y registrar cuánto tarda cada parte	Con Qwen3-14B el arranque puede consumir la mayor parte del trabajo	It. 4 y 6
RF3	Controlar la longitud de entrada y registrar la longitud real en tokens que cuenta el servidor	La longitud de entrada es una variable experimental y debe conocerse en tokens, no en caracteres	SPEC; it. 8
RF4	Enviar las solicitudes en streaming y registrar marcas de tiempo de alta resolución por fragmento en el cliente	Permite calcular TTFT, TPOT e ITL desde la perspectiva del usuario	SPEC
RF5	Recoger las métricas internas del servidor: `/metrics` en vLLM y los campos de duración en Ollama	Sostienen el tercer método de la triangulación	SPEC; it. 8
RF6	Muestrear en paralelo la telemetría de la GPU y registrar el load average del nodo	Sostiene la estimación de energía y el control de la contención	SPEC
RF7	Añadir un prefijo único a cada solicitud	Impide que el servidor reutilice el KV cache de la solicitud anterior y acorte artificialmente el prefill	It. 8
RF8	Reiniciar el servidor al cambiar de modelo y verificar que el modelo cabe completo en la GPU	Un servidor con memoria libre desactualizada puede repartir el modelo entre GPU y CPU	It. 8
RF9	Persistir cada corrida en JSON con sus metadatos: nodo, trabajo, versiones, modelo, parámetros, semilla y marca de tiempo, guardando tras cada solicitud	Hace comparables mediciones separadas por días y conserva los datos si el trabajo se interrumpe	SPEC; it. 2
RF10	Agregar los resultados y generar automáticamente las tablas para la memoria	Elimina la transcripción manual de resultados	SPEC

Elaborado por: José Andrés Ureta Granda
Fuente: Especificación del proyecto (`SPEC.md`, §11.1) e iteraciones de la Tabla 7

2.5.2 Requerimientos No Funcionales

Tabla 11: Requerimientos no funcionales del banco de pruebas.

ID	Requerimiento	Razón
RNF1 Reproducibilidad	Versiones y revisiones del modelo fijadas, semillas fijas, dependencias congeladas y scripts idempotentes	El primer objetivo específico exige un entorno reproducible
RNF2 Aislamiento de la GPU	GPU completa y dedicada, sin otros procesos durante la medición, verificada con `nvidia-smi`	Otro proceso en la GPU contaminaría latencia y energía
RNF3 Contención registrada	La carga de CPU del nodo, que no puede aislarse, se registra como covariable en cada corrida	Con load averages de hasta 790 no es realista exigir un nodo sin carga (Sección 2.4.2)
RNF4 Precisión de medición	Reloj monotónico (`time.perf_counter`), calentamiento descartado y repeticiones suficientes	Un reloj ajustable o una primera ejecución en frío sesgarían el TTFT
RNF5 Trazabilidad	Cada tabla o figura de la memoria referencia el identificador de la corrida y del trabajo de origen	Permite auditar cualquier cifra hasta su dato crudo
RNF6 Costo acotado	Cada fase tiene un tiempo máximo y la campaña cabe en el presupuesto de horas de GPU	Con un único trabajo simultáneo, el tiempo de GPU es el recurso escaso; las iteraciones 2, 4 y 6 lo agotaron sin medir
RNF7 Portabilidad del cliente	El cliente depende solo de la biblioteca estándar de Python	Puede ejecutarse en el nodo sin instalar nada, incluso cuando el entorno de vLLM no está disponible

Elaborado por: José Andrés Ureta Granda
Fuente: Especificación del proyecto (`SPEC.md`, §11.2) e iteraciones de la Tabla 7

El requerimiento RNF1 es el que más condiciona el diseño, porque obliga a congelar un entorno que el clúster no ofrece listo para usar, y el RF1 obliga a definir cómo se comunican el cliente y el servidor. La Sección 2.6 describe ambas cosas.

2.6 Diseño de la Solución

El diseño de la solución comprende tres elementos: la conexión entre el cliente de medición y el servidor de inferencia, el entorno de software congelado sobre el que se ejecutan todas las mediciones y el parche que hace posible ese entorno en los nodos del clúster.

2.6.1 Conexión con el servidor de inferencia

Los modelos no se ejecutan como una biblioteca dentro del programa de medición, sino detrás de un servidor de inferencia al que el cliente se conecta por HTTP. Se eligió esa arquitectura porque es la forma en que los modelos se sirven en producción, de modo que las latencias medidas incluyen los mismos componentes que percibiría un usuario real: la recepción de la solicitud, la planificación, el prefill, el decode y el envío de cada token.

La Figura 1 muestra la disposición. El investigador accede por SSH al nodo de acceso del clúster y envía el trabajo a Slurm. Dentro del trabajo, en el mismo nodo de GPU, se ejecutan dos procesos: el servidor, que carga el modelo en la A100 y escucha únicamente en la interfaz local `127.0.0.1`, y el cliente, que le envía las solicitudes. Al compartir nodo, la red no interviene en la medición: en la iteración 8, el tiempo atribuible al transporte y al cliente fue de alrededor de 1 ms. Escuchar solo en la interfaz local evita, además, exponer el servidor a otros usuarios del clúster.

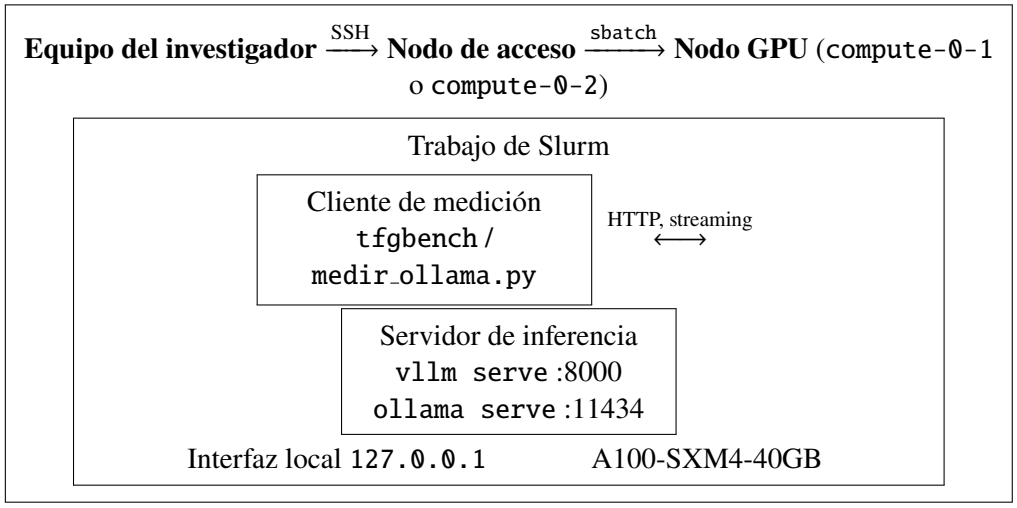


Figura 1: Conexión entre el cliente de medición y el servidor de inferencia.

Elaborado por: José Andrés Ureta Granda
Fuente: Elaboración propia

La Tabla 12 resume los puntos de acceso que usa el cliente con cada servidor. vLLM expone una API compatible con la de OpenAI, de modo que la solicitud de generación es un `POST` a `/v1/completions` con el modelo, el prompt, el número máximo de tokens y la opción de streaming. Ollama expone su API nativa en `/api/generate`; con ella se usa el modo `raw`, que envía el prompt sin aplicar la plantilla de conversación del modelo, para que el número de tokens de entrada dependa solo del texto enviado.
Elaborado por: José Andrés Ureta Granda

Tabla 12: Puntos de acceso de los servidores de inferencia utilizados por el cliente.

Función	vLLM 0.20.0 (puerto 8000)	Ollama 0.24.0 (puerto 11434)
Comprobar que está listo	`GET /health` (código 200)	`GET /api/version`
Identificar el modelo	`GET /v1/models`	`POST /api/show`
Generar en streaming	`POST /v1/completions` con `stream: true`	`POST /api/generate` con `stream: true` y `raw: true`
Métricas del servidor	`GET /metrics` (formato Prometheus)	Campos `prompt_eval_*`, `eval_*` y `load_duration` de la última respuesta

Fuente: Scripts `fase2_arranque.sh` y `medir_ollama.py`; documentación de vLLM y Ollama

El protocolo de una solicitud es el mismo con ambos servidores. El cliente fija la temperatura en 0 y la semilla en 0 para que la generación sea determinista, y fija el número de tokens de salida. Toma el instante inicial con un reloj monotónico, envía la solicitud y lee la respuesta fragmento a fragmento: el primer fragmento con texto marca el TTFT, y la separación entre fragmentos sucesivos da el ITL. Al terminar, el cliente guarda tanto sus propias marcas como las métricas que reporta el servidor, de modo que cada solicitud aporta datos a los métodos 2 y 3 de la triangulación. Antes de medir, el cliente envía una solicitud mínima que obliga a cargar el modelo y cuya duración se registra aparte como tiempo de carga, para que ese costo no contamine la primera medición.

2.6.2 Entorno de software congelado

La Tabla 13 recoge los componentes del entorno y sus versiones exactas. Se congelaron las revisiones de los modelos y el commit de la rueda de vLLM, y no solo sus números de versión, porque un mismo número de versión puede corresponder a binarios o pesos distintos. Las dependencias completas quedan en el archivo `req-congelado.txt`, que acompaña al repositorio. El entorno se construyó como un entorno Python versionado, una de las tres vías de reconstrucción que admite la ficha junto con Docker y Apptainer.

Elaborado por: José Andrés Ureta Granda

Fuente: Entorno instalado en el clúster HPC CEDIA

La caché de modelos se usa en modo sin conexión para que ningún trabajo descargue pesos durante la medición: la revisión que se ejecuta es siempre la que quedó registrada en la tabla. La razón de cada versión, en particular la de vLLM 0.20.0 y la del backend FlashAttention 2, se expone en la Sección 2.7. Antes es necesario resolver un problema que impedía cargar esa versión de vLLM en los

Tabla 13: Entorno de software congelado.

Componente	Versión
Python	3.11 (módulo `python/3.11`, miniconda)
PyTorch	2.11.0+cu129
CUDA runtime	12.9
vLLM	0.20.0+cu129
Procedencia de la rueda	`wheels.vllm.ai`, commit `88d34c6409e9fb3c7b8ca0c04756f061d2099eb1`
Backend de atención	FlashAttention 2
Modelo principal	`Qwen/Qwen3-14B`, revisión `40c069824f4251a91eefaf281ebe4c544efd3e18`
Modelo piloto	`Qwen/Qwen3-8B`, revisión `b968826d9c46dd6066d109eabc6255188de91218`
Caché de modelos	`HF_HOME=~/tfg-jose-ureta/hf-cache`, en modo sin conexión (`HF_HUB_OFFLINE=1`)
Servidor de contraste	Ollama 0.24.0 con la capa de compatibilidad CUDA 12.8
Dependencias completas	`req-congelado.txt`

Tabla 14: Bibliotecas del entorno con dependencias de glibc 2.32 o superior.

Biblioteca	¿Se carga en el clúster?
`vllm/_C.abi3.so`	Sí: núcleo de vLLM
`vllm/_moe_C.abi3.so`	Sí
`vllm/vllm_flash_attn/_vllm_fa3_C.abi3.so`	No: FlashAttention 3 es exclusiva de Hopper
`vllm/third_party/deep_gemm/_C.cpython-312-*.so`	No: compilada para Python 3.12

nodos.

2.6.3 Parche de compatibilidad glibc

La rueda `vllm-0.20.0+cu129` se distribuye con la etiqueta `manylinux_2_31`, que declara compatibilidad con glibc 2.31 o superior [25]. Sin embargo, dos de sus extensiones nativas, `_C.abi3.so` y `_moe_C.abi3.so`, enlazan cuatro símbolos con la versión `GLIBC_2.34`: `dlopen`, `dlclose`, `dlsym` y `pthread_once`. En los nodos, con glibc 2.31, el resultado es un `ImportError` al cargar vLLM. La etiqueta de la rueda es, por tanto, incorrecta. Este fue el primer fallo de la Tabla 7.

El diagnóstico consistió en inspeccionar con `objdump -T` todas las bibliotecas compartidas del entorno virtual. Solo cuatro declaran dependencias de glibc 2.32 o superior, y la Tabla 14 indica cuáles se cargan efectivamente en el clúster. Las dos que no se cargan quedan fuera del problema: una implementa FlashAttention 3, que solo existe para Hopper, y la otra está compilada para Python 3.12. El parche debe actuar, en consecuencia, solo sobre las dos extensiones del núcleo de vLLM.

Elaborado por: José Andrés Ureta Granda

Fuente: Inspección con `objdump -T` del entorno virtual

Los cuatro símbolos no corresponden a funcionalidad ausente en glibc 2.31. En glibc 2.34 las bibliotecas `libdl` y `libpthread` se consolidaron dentro de `libc`; en glibc 2.31 esos mismos símbolos existen, pero residen en `libdl.so.2` y `libpthread.so.0`.[3] La incompatibilidad es, por tanto, un artefacto de haber compilado la rueda en un sistema más moderno, y basta con indicar al cargador dinámico dónde encontrar los símbolos.

La solución se implementó en el script `parche_glibc.sh`, que aplica tres operaciones. Primero, elimina la versión de los cuatro símbolos con `patchelf --clear-symbol-version` y añade `libdl.so.2` y `libpthread.so.0` como dependencias explícitas con `--add-needed`.[4] Segundo, marca como `VER_FLG_WEAK` la entrada `GLIBC_2.34` de la sección `.gnu.version_r` mediante `parche_glibc_weak.py`, porque el paso anterior no basta: el cargador sigue rechazando la biblioteca mientras esa entrada exista sin la bandera. Tercero, verifica que `import vllm._C` e `import vllm._moe_C` funcionen. Que hicieran falta dos pasos, y no uno, es otro ejemplo del método: el primer intento con `patchelf` no bastó, y fue el mensaje del cargador el que indicó el segundo ajuste.

El script es idempotente y conserva los binarios originales como `*.so.orig`, de modo que aplicarlo varias veces no altera el resultado y el cambio es reversible. Debe reaplicarse tras cada reinstalación del entorno virtual, y por eso los scripts de medición lo invocan automáticamente si detectan que vLLM no se importa. Así, ningún trabajo de medición depende de que el parche se haya aplicado a mano.

El parche no altera el comportamiento numérico de vLLM. Solo reubica dónde busca el cargador dinámico cuatro símbolos de carga de bibliotecas y sincronización; no modifica kernels, precisión numérica ni planificación de solicitudes. Aun así, constituye una modificación del binario distribuido, y el entorno resultante no es bit a bit idéntico al que se obtendría con `pip install vllm==0.20.0` en un sistema con glibc 2.34 o superior. Por esa razón, los scripts del parche acompañan al repositorio, para que cualquier persona pueda reproducir exactamente el mismo entorno. Las alternativas que se evaluaron antes de optar por el parche se discuten en la Sección 2.7.

[3] GNU C Library, notas de la versión 2.34 (archivo `NEWS`, apartado *Major new features*): toda la funcionalidad de `libpthread`, `libdl`, `libutil` y `libanl` se integró en `libc`. Disponible en `https://sourceware.org/git/?p=glibc.git;a=blob_plain;f=NEWS;hb=glibc-2.34`. Consultado el 24 de octubre de 2026.

[4] patchelf, utilidad para modificar ejecutables y bibliotecas ELF; ambas opciones se describen en su documentación y su código fuente, disponibles en `https://github.com/NixOS/patchelf`. Consultado el 24 de octubre de 2026.

2.7 Justificación Técnica

La justificación técnica parte de las tecnologías que fija el caso de estudio: el motor de inferencia vLLM, que gestiona el KV cache mediante PagedAttention y planifica la ejecución a nivel de iteración [4]; la familia de modelos Qwen3 [2], en sus tamaños de 14 y 8 mil millones de parámetros; y la GPU NVIDIA A100 [3]. Esta sección justifica las decisiones que el trabajo tomó dentro de ellas: las versiones, el backend de atención, la vía de compatibilidad y la factibilidad de la matriz en memoria.

2.7.1 Versiones y backend de atención

La versión 0.20.0 de vLLM no se eligió por preferencia, sino por una restricción del entorno. Las versiones posteriores distribuyen su stack compilado contra CUDA 13, y el clúster operaba con el driver 535 en el momento de la instalación. La compatibilidad de versiones menores de CUDA permite ejecutar runtimes 12.x sobre drivers 525 o superiores, pero exige un driver 580 o superior para CUDA 13, de modo que no cruza de la versión mayor 12 a la 13.[5] La versión 0.20.0 con CUDA runtime 12.9 es, por tanto, la más reciente ejecutable con aquel driver sin recurrir a la compatibilidad hacia adelante, el mecanismo de NVIDIA que permite ejecutar versiones de CUDA más recientes sobre drivers anteriores en GPU de centro de datos mediante el paquete `cuda-compat`. La misma restricción decidió la versión de Ollama: se usó la 0.24.0 con la capa de compatibilidad CUDA 12.8 porque las versiones a partir de la 0.30 exigen un driver 550 o superior.

El driver 575.51.03 observado en `compute-0-2`, recogido en la Tabla 5, no invalida esta decisión. Ese driver sigue por debajo de 580, de modo que la compatibilidad de versiones menores tampoco permitiría ejecutar CUDA 13 con él. Mantener la versión congelada preserva, además, la comparabilidad entre el piloto con Qwen3-8B y la campaña con Qwen3-14B.

vLLM selecciona FlashAttention-2 [26] como backend de atención, y el registro del arranque de Qwen3-14B lo confirma, porque FlashAttention-3 está diseñada para explotar características de la arquitectura Hopper [27], y la A100 pertenece a la arquitectura Ampere. FlashAttention reduce los accesos a la memoria de alto ancho de banda de la GPU durante el cálculo de la atención [9]. Se trata de una limitación

[5] NVIDIA, *CUDA Toolkit Release Notes*, tabla de versiones mínimas del driver para la compatibilidad de versiones menores, disponible en `https://docs.nvidia.com/cuda/cuda-toolkit-release-notes/`; y *CUDA Compatibility*, apartado de compatibilidad hacia adelante, disponible en `https://docs.nvidia.com/deploy/cuda-compatibility/forward-compatibility.html`. Consultado el 24 de octubre de 2026.

de la plataforma y no de una decisión del experimento. Se declara porque afecta al rendimiento absoluto de la fase de prefill, que es la más intensiva en la atención.

Para resolver la incompatibilidad con glibc se evaluaron tres alternativas antes de optar por el parche, recogidas en la Tabla 15. El contenedor oficial queda como plan B porque resuelve el problema sin modificar binarios, pero su tamaño lo hace costoso en el almacenamiento compartido, un costo que la iteración 6 confirmó al medir lo que tarda leer el entorno desde el NFS. La compilación desde el código fuente se descartó por su costo en tiempo, y la regresión a una versión anterior, porque no resolvía el problema.

Tabla 15: Alternativas evaluadas para resolver la incompatibilidad con glibc 2.31.

Alternativa	**Motivo del descarte**
Contenedor oficial `vllm/vllm-openai` con enroot/pyxis o Apptainer	Imagen superior a 10 GB sobre NFS compartido; queda como plan B
Compilar vLLM desde el código fuente en el nodo	Varias horas de compilación por cada reinstalación
Fijar una versión anterior de vLLM	No resuelve el problema: la etiqueta incorrecta aparece en varias versiones

Elaborado por: José Andrés Ureta Granda
Fuente: Elaboración propia

Fijadas las versiones, queda por justificar que la plataforma puede alojar la matriz experimental y qué rendimiento cabe esperar de ella como máximo. La Sección 2.7.2 lo establece mediante cálculos analíticos para los dos modelos.

2.7.2 Techos teóricos y presupuesto de memoria

Los techos teóricos sirven de referencia contra la cual se contrastarán las mediciones. Se calculan con el modelo roofline, que acota el rendimiento de un cálculo por el menor de dos límites: el cómputo máximo del procesador y el ancho de banda de la memoria [19]. Sin esta referencia, una cifra de throughput es un número suelto; con ella, indica qué fracción del límite físico alcanza cada fase.

El punto de partida es el tamaño de los pesos. Qwen3-14B tiene aproximadamente $N = 14{,}8 \cdot 10^9$ parámetros y Qwen3-8B, $N = 8{,}2 \cdot 10^9$ [2]. En BF16 cada parámetro ocupa 2 bytes, y la Ecuación 12 obtiene el volumen de los pesos. Para Qwen3-14B el resultado, 27,6 GiB, coincide con los 27,52 GiB que el motor registró al cargar el modelo en la iteración 6; para Qwen3-8B equivale a 15,3 GiB.

$$M_{pesos} = N \times 2\ \text{B} \approx \begin{cases} 29{,}6\ \text{GB} & \text{Qwen3-14B} \\ 16{,}4\ \text{GB} & \text{Qwen3-8B} \end{cases} \tag{12}$$

En el decode con lote 1, cada token generado exige leer todos los pesos desde la memoria HBM2 de la GPU, y el cómputo por byte leído es bajo. La fase queda entonces limitada por el ancho de banda de memoria, y su techo se obtiene con la Ecuación 13. El cálculo ignora la lectura del KV cache, que con lote 1 es pequeña frente a los pesos, de modo que el techo real es algo inferior. Expresado como tiempo por token, el TPOT de Qwen3-14B no puede bajar de unos 19 ms, y el de Qwen3-8B, de unos 10,5 ms.

$$X^{max}_{decode} \approx \frac{BW_{HBM}}{M_{pesos}} \approx \begin{cases} \dfrac{1555\ \text{GB/s}}{29{,}6\ \text{GB}} \approx 53\ \text{tok/s} & \text{Qwen3-14B} \\ \dfrac{1555\ \text{GB/s}}{16{,}4\ \text{GB}} \approx 95\ \text{tok/s} & \text{Qwen3-8B} \end{cases} \tag{13}$$

En el prefill, en cambio, todos los tokens del prompt se procesan en paralelo y los pesos leídos se reutilizan para cada uno de ellos. La fase queda limitada por el cómputo, a razón de aproximadamente dos operaciones de coma flotante por parámetro y token, una multiplicación y una suma. La Ecuación 14 da su techo con el rendimiento BF16 denso de la A100.

$$X^{max}_{prefill} \approx \frac{P_{BF16}}{2N} \approx \begin{cases} \dfrac{312 \cdot 10^{12}\ \text{FLOP/s}}{2 \times 14{,}8 \cdot 10^{9}\ \text{FLOP/tok}} \approx 10\,500\ \text{tok/s} & \text{Qwen3-14B} \\ \dfrac{312 \cdot 10^{12}\ \text{FLOP/s}}{2 \times 8{,}2 \cdot 10^{9}\ \text{FLOP/tok}} \approx 19\,000\ \text{tok/s} & \text{Qwen3-8B} \end{cases} \tag{14}$$

En ambos modelos los dos techos difieren en dos órdenes de magnitud, y esa diferencia es la expresión cuantitativa del argumento central del trabajo: prefill y decode son fases con cuellos de botella distintos. Contrastar las mediciones contra estos techos es lo que permitirá identificar el cuello de botella dominante de cada fase, como exige el cuarto objetivo específico. Comparar los dos modelos añade una predicción verificable: si el decode está limitado por la memoria, el TPOT debe crecer aproximadamente en la misma proporción que el volumen de los pesos, un factor cercano a 1,8.

El presupuesto de memoria determina qué configuraciones caben en la GPU.

Los parámetros de arquitectura se toman del archivo `config.json` de la revisión efectivamente descargada, porque es esa revisión la que se ejecuta. Ambos modelos usan 8 cabezas KV de dimensión 128, gracias a la atención de consultas agrupadas (GQA) [20], y difieren en el número de capas: 40 en Qwen3-14B y 36 en Qwen3-8B [2]. La Ecuación 15 calcula el KV cache por token, donde el factor 2 inicial corresponde a las claves y los valores, y los 2 bytes finales, a la representación BF16.

$$m_{KV} = 2\, n_{capas}\, n_{KV}\, d_h\, b = \begin{cases} 2 \times 40 \times 8 \times 128 \times 2\ \text{B} = 160{,}0\ \text{KiB} & \text{Qwen3-14B} \\ 2 \times 36 \times 8 \times 128 \times 2\ \text{B} = 144{,}0\ \text{KiB} & \text{Qwen3-8B} \end{cases} \tag{15}$$

La diferencia decisiva no está en el KV cache por token, que solo crece un 11 %, sino en el espacio que queda para él. Para el piloto con Qwen3-8B, la configuración más exigente de la ficha combina 8192 tokens con 16 secuencias concurrentes. Las Ecuaciones 16 y 17 calculan su KV cache y le suman los pesos: el total, 33,3 GiB, queda dentro de los aproximadamente 36 GiB que vLLM puede usar con `--gpu-memory-utilization 0.90`, aunque con un margen estrecho.

$$M_{KV} = 8192 \times 16 \times 147\,456\ \text{B} \approx 18{,}0\ \text{GiB} \tag{16}$$

$$M_{total} = M_{pesos} + M_{KV} \approx 15{,}3\ \text{GiB} + 18{,}0\ \text{GiB} = 33{,}3\ \text{GiB} \tag{17}$$

Ese presupuesto omitía los grafos CUDA, que en el piloto ocuparon 2,44 GiB, según muestra la Sección 3.3.3, y con ellos la configuración ya no cabía. Con Qwen3-14B el margen es mucho menor. La Ecuación 18 resta de los 36 GiB los pesos medidos y, suponiendo que los grafos CUDA ocupen lo mismo que en el piloto, obtiene el espacio que queda para el KV cache y su equivalente en tokens.

$$M_{KV}^{disp} \approx 36\ \text{GiB} - 27{,}5\ \text{GiB} - 2{,}4\ \text{GiB} \approx 6{,}1\ \text{GiB} \;\Rightarrow\; \frac{6{,}1\ \text{GiB}}{160\ \text{KiB}} \approx 40\,000\ \text{tokens} \tag{18}$$

La Tabla 16 aplica el cálculo a las configuraciones de la ficha y a las de los experimentos E1 y E2. La columna intermedia muestra la cota sin descontar los

grafos CUDA, unos 8,5 GiB, para indicar qué configuraciones dependen de ese descuento.

Tabla 16: Factibilidad en memoria de las configuraciones con Qwen3-14B.

Configuración	**Tokens en caché**	**KV cache**	**¿Cabe en 8,5 GiB?**	**¿Cabe en 6,1 GiB?**
E1: 7680 + 128 tokens, concurrencia 1	7 808	1,2 GiB	Sí	Sí
E2: 1280 tokens, concurrencia 16	20 480	3,1 GiB	Sí	Sí
E2: 1280 tokens, concurrencia 32	40 960	6,3 GiB	Sí	No, por poco
E2: 1280 tokens, concurrencia 64	81 920	12,5 GiB	No	No
Ficha: 8192 tokens, concurrencia 16	131 072	20,0 GiB	No	No

Elaborado por: José Andrés Ureta Granda

Fuente: Elaboración propia a partir de las Ecuaciones 15 y 18

El cálculo indica que, con Qwen3-14B, la A100 de 40 GB admite unas cinco secuencias concurrentes de 8192 tokens, frente a las 16 de la ficha, de modo que la matriz no es ejecutable tal como está y debe ajustarse, que es lo que prevé el riesgo R6. Los puntos de E2 que no caben no se eliminan: en ellos vLLM deberá dejar solicitudes en espera o expulsarlas de la caché y recomputarlas, y medir ese comportamiento es en sí un resultado sobre el límite de la plataforma. La cifra de 40 000 tokens es una estimación; el valor real lo reporta vLLM al arrancar, y su registro es una de las mediciones pendientes de la fase 2. Con el método, los instrumentos, el entorno y las cotas analíticas establecidos, el Capítulo III describe la implementación del banco de pruebas y la ejecución de las iteraciones.

CAPÍTULO III

IMPLEMENTACIÓN, PRUEBAS Y RESULTADOS

3.1 Descripción de la Solución

La solución implementada es un banco de pruebas que mide por separado las fases prefill y decode de un modelo servido en una A100-SXM4-40GB del clúster HPC CEDIA. Su objetivo es la campaña con Qwen3-14B servido con vLLM, y se construyó y calibró con Qwen3-8B como piloto. El banco no se diseñó de una vez: cada componente apareció como respuesta a un fallo o a una limitación observada en una iteración concreta de la Tabla 7. La Tabla 17 recoge los componentes, su función y la iteración que los originó.

Tabla 17: Componentes del banco de pruebas.

Componente	Función	Iteración
Entorno congelado y `parche_glibc.sh`	Ejecutar vLLM 0.20.0 en los nodos con glibc 2.31 (Sección 2.6)	1
`bench_pd.sh`	Barrido sin servidor con `vllm bench latency`, un motor por configuración	2
Cliente `tfgbench`	Cliente en streaming contra un servidor único; salida compatible con `vllm bench serve`	3
`a100_qwen3_8b.sh`	Trabajo de Slurm con un único `vllm serve` y los experimentos E1 y E2	3 y 4
`ensayo.sh`	Ensayo del arranque de Qwen3-14B desde el NFS con carga mínima	6
`activar_scratch.sh`, `fase1_preparar.sh`	Copia del entorno virtual y de la caché de modelos al disco local del trabajo	7
`fase2_arranque.sh`	Arranque cronometrado de `vllm serve` desde `/scratch`, con espera máxima de 15 min	7
`medir_ollama.py`, `resumir.py`	Medición de contraste contra Ollama y agregación de resultados en JSON y Markdown	8

Elaborado por: José Andrés Ureta Granda

Fuente: Repositorio del trabajo y carpeta `tfg-jose-ureta` del clúster

Este capítulo sigue el mismo orden. La Sección 3.2 describe cada iteración: qué se implementó, qué ocurrió al ejecutarlo y qué se cambió después. La Sección 3.3 reúne las pruebas que validan los instrumentos, y la Sección 3.4, los resultados obtenidos hasta la fecha: los del piloto con Qwen3-8B y los del contraste con Ollama. La campaña con Qwen3-14B no ha producido todavía mediciones de rendimiento,

porque el arranque del servidor es el problema abierto de la iteración 7; el capítulo lo declara en lugar de omitirlo.

3.2 Proceso de Implementación

3.2.1 Piloto con Qwen3-8B: trabajo 30732

El trabajo 30732, correspondiente a la iteración 2, se ejecutó el 24 de octubre de 2026 sobre una A100 completa, sin particiones MIG. La Tabla 18 recoge el entorno registrado por la fase 0 en `entorno.txt`. La carga del nodo al inicio, 37,20 en la ventana de un minuto sobre 256 núcleos, es baja frente a la documentada en la Sección 2.4.2.

Tabla 18: Entorno registrado en el trabajo 30732.

Elemento	Valor
Nodo	`compute-0-1`
Inicio	24 de octubre de 2026, 11:07:24 (UTC−5)
Carga al inicio (1, 5 y 15 min)	37,20; 36,72; 40,91
GPU	NVIDIA A100-SXM4-40GB, 40 960 MiB, límite de 400 W, 1410 MHz máx.
Driver	535.161.08
Software	vLLM 0.20.0, PyTorch 2.11.0+cu129, CUDA 12.9
Modelo	`Qwen/Qwen3-8B`, BF16
Opciones del motor	`max_model_len`=8192, `gpu_memory_utilization`=0,90, caché de prefijos desactivada, chunked prefill activo con 8192 tokens por lote, grafos CUDA y `torch.compile` activos

Elaborado por: José Andrés Ureta Granda
Fuente: `entorno.txt` y registros del trabajo 30732

Cada configuración del barrido sin servidor se midió con `vllm bench latency`, que crea un motor nuevo en cada invocación, ejecuta tres iteraciones de calentamiento y mide diez. La Tabla 19 desglosa la duración de cada ejecución. La columna de inicialización corresponde al tiempo que vLLM declara para perfilar la memoria, crear el KV cache y calentar el modelo, del cual la mayor parte es compilación con `torch.compile`.
Elaborado por: José Andrés Ureta Granda
Fuente: Registros `smoke.txt` y `lat_in*_out*_bs1.log` del trabajo 30732

La tabla muestra una desproporción que condiciona el resto de la implementación. Las tres configuraciones completadas sumaron menos de 18 s de medición efectiva en un trabajo de casi cinco horas. La primera compilación de

Tabla 19: Duración de las ejecuciones del trabajo 30732.

Ejecución	Inicio	Inicialización del motor (s)	Duración total (min)	Medición (s)
Prueba de humo (L_{in} máx. 2048)	11:32:35	632,8	15,9	—
L_{in} = 128, L_{out} = 1	12:11:47	1452,4	74,6	0,17
L_{in} = 128, L_{out} = 129	13:49:08	436,4	28,7	16,95
L_{in} = 512, L_{out} = 1	14:47:50	248,6	43,1	0,37
L_{in} = 512, L_{out} = 129	15:55:36	—	Incompleta	—

`torch.compile` tardó 1416 s y guardó su resultado en la caché `~/.cache/vllm/torch_compile_cache`, lo que explica que las inicializaciones siguientes bajaran a 436 s y 249 s. Aun así, cada ejecución siguió costando entre 29 y 75 minutos.

La inicialización del motor no explica toda esa duración. En la primera ejecución transcurrieron 38 minutos entre la lectura de la configuración, a las 12:12:25, y la creación del motor, a las 12:50:35, y entre el final de una ejecución y el primer registro de la siguiente pasaron 23 y 30 minutos. En el momento del piloto la causa de estas esperas no quedó determinada. Los pesos residen en un sistema de archivos NFS4, según el propio registro de vLLM, y su lectura tardó 57 s en la primera ejecución y 10 s en las siguientes, de modo que la lectura de los pesos tampoco basta para explicarlas. La iteración 6, descrita en la Sección 3.2.4, aportó la explicación más probable: la importación de la pila de Python desde el mismo NFS.

La quinta configuración no llegó a medirse. La telemetría se detiene a las 16:06:11, cuando el motor de esa ejecución aún se estaba inicializando, y ningún registro indica un error. Ese corte coincide con el límite de 5 horas del trabajo.

3.2.2 Servidor único y cliente propio

El trabajo 30732 demostró que, en esta plataforma, el costo dominante no es la medición sino el arranque del motor. Con un arranque de entre 29 y 75 minutos, un barrido que crea un motor por configuración no cabe en un trabajo de Slurm, por largo que sea su límite de tiempo. El ajuste de la iteración 3 fue levantar el servidor una sola vez y recorrer todas las configuraciones contra él, una decisión que pasa así de ser una optimización a ser una condición de viabilidad.

Para ello se implementó el cliente `tfgbench`. Recibe un archivo TOML con la URL del servidor, el modelo, el tokenizador con su revisión y los niveles de cada variable; genera prompts de longitud exacta en tokens con ese tokenizador; envía las solicitudes en streaming, en bucle cerrado con como mucho c solicitudes en vuelo

para una concurrencia c; y guarda un JSON por configuración con el mismo formato que `vllm bench serve`, de modo que los resultados de ambas herramientas son comparables. Los prompts son secuencias aleatorias de identificadores de token, generadas con semilla 0 y sin prefijos compartidos, y las solicitudes se envían con temperatura 0 e `ignore_eos`.

El cliente se validó primero con un modelo pequeño en una partición MIG de 5 GB, que el planificador asigna sin espera. La Tabla 20 recoge ese piloto con Qwen3-0.6B. Sus cifras no se usan como resultados, porque el modelo y la GPU no son los del estudio, pero muestran que el cliente produce métricas coherentes: el TTFT crece con la entrada y con la concurrencia, y el TPOT crece con la concurrencia, como anticipa la Sección 1.7.2.

Tabla 20: Validación del cliente `tfgbench` con Qwen3-0.6B en una partición MIG de 5 GB.

Entrada (tokens)	Concurrencia	TTFT p50 (ms)	TPOT p50 (ms)	ITL p99 (ms)	Salida (tok/s)
128	1	18,9	8,3	11,0	118,5
128	4	51,3	9,7	11,1	400,5
1024	1	57,6	8,9	10,3	107,6
1024	4	183,9	12,4	14,9	288,9

Elaborado por: José Andrés Ureta Granda
Fuente: Corrida `piloto-mig-0.6b-20261027-155013`: salida 128 tokens, 8 solicitudes medidas y 2 de calentamiento por punto

El script `a100_qwen3_8b.sh` integra el servidor y el cliente en un trabajo de Slurm. Solicita una A100 completa en la partición `gpu-dev`, levanta un único `vllm serve` con las mismas opciones de la Tabla 18, espera hasta 45 minutos a que el endpoint `/health` responda y ejecuta después los experimentos con `tfgbench` en el mismo nodo, contra `127.0.0.1`, según la disposición de la Sección 2.6.1. Tras cada experimento guarda las métricas acumuladas de `/metrics`, que sostienen el tercer método de la triangulación de la Sección 2.4.1, y reduce el periodo de muestreo de la telemetría de 2 s a 1 s. La Tabla 21 resume los dos experimentos que ejecuta.
Elaborado por: José Andrés Ureta Granda
Fuente: `configs/e1_prefill.toml` y `configs/e2_concurrencia.toml`

La longitud máxima de E1 es 7680 y no 8192, porque la suma de entrada y salida no puede superar el `max_model_len` de 8192. En E2 con Qwen3-8B, 64 secuencias de 1280 tokens ocupan unos 11,3 GiB de KV cache y caben sin expulsiones en los 16,3 GiB disponibles, según la Sección 3.3.3. Con Qwen3-14B los mismos puntos se conservan, pero los de concurrencia 32 y 64 superan la memoria disponible, como

Tabla 21: Experimentos E1 y E2.

Parámetro	E1: prefill	E2: concurrencia
Longitud de entrada	128, 256, 512, 1024, 2048, 4096 y 7680	1024
Longitud de salida	128	256
Concurrencia	1	1, 2, 4, 8, 16, 32 y 64
Solicitudes medidas por punto	30 (más 3 de calentamiento)	256 (más 4 de calentamiento)
Métricas principales	TTFT	TPOT, ITL y throughput

calcula la Tabla 16. Para el modelo mayor se preparó un TOML equivalente que solo cambia el modelo y la revisión del tokenizador.

El primer lanzamiento de `a100_qwen3_8b.sh`, el trabajo 31157 del 27 de octubre, constituyó la iteración 4 y no llegó a medir. La preparación del entorno tardó 21 min 44 s, entre las 17:11:06 y las 17:32:50, y el servidor no respondió en los 45 minutos de espera siguientes; su registro quedó vacío y la GPU no llegó a ocupar memoria. Sin registro del servidor no fue posible diagnosticar la causa, y el ajuste fue el que recoge la Tabla 7: medir el tiempo de cada parte del arranque en los intentos siguientes.

3.2.3 Cambio a Qwen3-14B y diagnóstico de la sesión interactiva

El 29 de octubre, por indicación del tutor, el modelo principal pasó a ser Qwen3-14B en BF16. Se descargó la revisión `40c06982` a la caché de modelos y se preparó una versión del script de campaña para el nuevo modelo. Antes de lanzarla se intentó arrancar el servidor a mano en una sesión interactiva, el trabajo 31299, para observar el arranque en directo. Esa sesión constituye la iteración 5.

La sesión obtuvo una A100 completa en `compute-0-2`, pero tras los primeros comandos la shell dejó de responder, incluso a órdenes triviales como `echo`. La lectura directa de `/proc` mostró un load average de 163,88 con solo 8 CPU asignadas al trabajo, una presión de CPU del 21 % y ninguna presión de disco ni de memoria, de modo que no se trataba de un sistema de archivos colgado. La causa identificada fue el archivo `~/.bashrc`, que ejecutaba `spack load hwloc` en cada shell nueva; con el nodo sobrecargado, esa orden bloqueaba la apertura de cada shell. Se retiró del `.bashrc`, y la iteración dejó una regla para el resto del trabajo: los scripts de medición no dependen de la configuración interactiva del usuario.

3.2.4 Arranque de Qwen3-14B desde el NFS

La iteración 6, el 2 de noviembre, aisló el arranque. El script `ensayo.sh` levanta `vllm serve` con Qwen3-14B y las mismas opciones del piloto, espera hasta 90 minutos a `/health` y envía después una carga mínima de dos solicitudes. El entorno virtual y la caché de modelos se leyeron, como en el piloto, desde el NFS. La Tabla 22 reconstruye el arranque a partir de `serve.log`.

Tabla 22: Cronología del arranque de `vllm serve` con Qwen3-14B desde el NFS (2 de noviembre).

Hora	Transcurrido	Evento
09:08:51	0:00	Lanzamiento de `vllm serve`
09:28:20	19:29	Primera línea del registro: versión de vLLM y argumentos
09:54:46	45:55	Inicio del proceso del motor (`EngineCore`)
09:59:15	50:24	Comienza la carga del modelo
10:01:20	52:29	Selección del backend FlashAttention 2
10:02:11	53:20	vLLM detecta que el checkpoint, de 27,51 GiB, reside en NFS4 y lo precarga en la caché de páginas
10:03:46	54:55	Precarga terminada en 95,08 s
10:04:02	55:11	Pesos cargados en 110,48 s; el modelo ocupa 27,52 GiB de la GPU tras 285,9 s de carga
10:08:22	59:31	`TimeoutError`: se superan los 600 s de `VLLM_ENGINE_READY_TIMEOUT_S`

Elaborado por: José Andrés Ureta Granda

Fuente: `serve.log` y `ensayo.out` del ensayo `linea-base-home-20261102`

La cronología separa dos costos de naturaleza distinta. Los primeros 46 minutos transcurren antes de que exista el proceso del motor, sin tocar la GPU ni los pesos: corresponden a la importación de vLLM, PyTorch y sus dependencias, miles de archivos pequeños leídos desde el NFS por un nodo con alta carga. La lectura de los pesos, en cambio, es un único bloque de 27,5 GiB que el NFS sirvió en menos de dos minutos. El servidor llegó a tener el modelo en la GPU, pero el motor no completó la compilación y la captura de grafos dentro de los 600 s que vLLM concede por defecto, y el arranque se abandonó a los 59 min 31 s.

El diagnóstico desplaza el problema: no está en la GPU ni en el tamaño del modelo, sino en el número de archivos del entorno y en dónde residen. Ese diagnóstico explica también, de forma retrospectiva, las esperas de 23 a 38 minutos sin registro del piloto 30732 y el registro vacío del trabajo 31157.

3.2.5 Copia del entorno al disco local

El ajuste de la iteración 7 fue ejecutar el servidor desde el disco local del nodo. El script `activar_scratch.sh` copia la caché de modelos y el entorno virtual a `/scratch/$USER/$SLURM_JOB_ID`, adapta las rutas del entorno virtual a su nueva ubicación y lo activa desde la copia; `fase1_preparar.sh` lo invoca y verifica que `import vllm` se resuelva desde `/scratch`. Los registros se escriben en el directorio personal, porque `/scratch` se borra al terminar el trabajo. La segunda fase, `fase2_arranque.sh`, cronometra el arranque desde la copia con las mismas opciones del ensayo, espera hasta 15 minutos a `/health` y, si se excede, no detiene el servidor: lo informa y deja la decisión al investigador.

La fase 1 se ejecutó dos veces, en ambos casos en `compute-0-1`. En el trabajo 32710, del 8 de noviembre, la caché de modelos se copió en 459 s, y el registro se interrumpe durante la copia del entorno virtual. En el trabajo 33107, del 11 de noviembre, la caché se copió en 414 s, pero del entorno virtual, de unos 12 GB, solo se habían copiado 1,2 GB a las 18:02, 71 minutos después de empezar, con un load average cercano a 790. La copia confirma el diagnóstico de la iteración 6 por otra vía: los pesos, pocos archivos grandes, se copian en unos 7 minutos, mientras que el entorno, muchos archivos pequeños, no avanza a un ritmo útil. La fase 2 no llegó a ejecutarse.

3.2.6 Contraste con Ollama

Mientras la copia del entorno avanzaba, y dentro del mismo trabajo 33107, la iteración 8 midió un segundo servidor de inferencia, Ollama 0.24.0, para disponer de un contraste independiente de vLLM. Ollama se instaló en el directorio personal con la capa de compatibilidad CUDA 12.8, porque sus versiones más recientes exigen un driver superior al del nodo, y se ejecutó con una sola solicitud a la vez (`OLLAMA_NUM_PARALLEL=1`), escuchando en `127.0.0.1:11434`.

El cliente `medir_ollama.py` usa solo la biblioteca estándar de Python, conforme al requerimiento RNF7, y se conecta a la API nativa de Ollama según la Tabla 12. Para cada modelo registra primero el nodo, el trabajo, las versiones, los detalles del modelo, la memoria de la GPU y el load average. Después envía una solicitud mínima que obliga a cargar el modelo y la cronometra aparte. Para cada longitud de entrada ejecuta una solicitud de calentamiento y cinco medidas, con 128 tokens de salida, temperatura 0, semilla 0 y contexto de 8192 tokens. Cada prompt comienza con un prefijo único, formado por el índice de la solicitud y una marca de tiempo, para que el servidor no reutilice el KV cache de la anterior, como exige el RF7. El resultado

se guarda en `datos.json` tras cada solicitud, de modo que una interrupción no pierde lo medido, y al final se agregan la mediana, la media, el percentil 90 y la desviación típica de cada métrica en `resumen.json`. El script `resumir.py` reúne todas las corridas en una tabla común.

La longitud de entrada se controla en palabras y no en tokens, porque el cliente no carga el tokenizador; la longitud real en tokens la reporta el servidor en cada respuesta y es la que se usa en el análisis, según el RF3. Se midieron dos modelos de 30 mil millones de parámetros disponibles en el registro de Ollama: `qwen3-coder:30b` y `nemotron-3-nano:30b`. El primero se midió completo; el segundo se invalidó, como explica la Sección 3.3.4.

3.3 Pruebas y Validación

3.3.1 Prueba de humo y verificación del dispositivo

La prueba de humo del trabajo 30732 cargó Qwen3-8B y generó 64 tokens de texto coherente en español a partir de un prompt sobre la inferencia de modelos de lenguaje. Su tiempo total, 13,77 s, incluye la primera ejecución del modelo y no se usa como medida de rendimiento. La prueba cumple su función: confirma que el entorno parcheado de la Sección 2.6.3 carga el modelo y genera texto antes de comprometer el resto del trabajo.

El dispositivo se verificó en todos los trabajos con `nvidia-smi -L`, que identifica una A100-SXM4-40GB completa y no una instancia MIG, conforme a la precaución de la Sección 2.1; en el trabajo 33107 lo confirmó también el propio Ollama, que detectó una GPU de capacidad de cómputo 8.0 con 40 GiB. El backend de atención seleccionado por vLLM fue FlashAttention-2 tanto con Qwen3-8B como con Qwen3-14B, como anticipa la Sección 2.7.1.

3.3.2 Estabilidad de las mediciones

La Tabla 23 resume la dispersión de las diez iteraciones medidas en cada configuración del piloto. El coeficiente de variación no supera el 0,6 %, y el rango completo de cada configuración es inferior a 1 ms. Con lote 1 y sin servidor, la medición es, por tanto, muy repetible dentro de un mismo trabajo.

Elaborado por: José Andrés Ureta Granda

Fuente: Archivos `lat_in*_out*_bs1.json` del trabajo 30732

Las iteraciones de calentamiento confirman la necesidad de descartarlas. La primera de cada configuración tardó 25,2 s, 14,9 s y 33,5 s, respectivamente, entre 9

Tabla 23: Dispersión de la latencia en las diez iteraciones medidas del piloto con Qwen3-8B.

Configuración	Media (ms)	Desv. típica (ms)	CV (%)	Mín. (ms)	Máx. (ms)
L_{in} = 128, L_{out} = 1	17,087	0,099	0,58	16,943	17,205
L_{in} = 512, L_{out} = 1	37,207	0,138	0,37	36,984	37,396
L_{in} = 128, L_{out} = 129	1695,221	0,306	0,02	1694,844	1695,809

y 1500 veces más que la media de las iteraciones medidas, porque incluye la captura de grafos y la carga de kernels pendientes. Incluirla en la media habría invalidado la medición, como prevé el requerimiento RNF4.

La baja dispersión no garantiza, sin embargo, la reproducibilidad entre trabajos. Las diez iteraciones de cada configuración se ejecutaron en menos de 20 s, dentro de un mismo motor y con una carga del nodo estable. La variación entre trabajos, nodos y versiones del driver, que la Sección 2.4.2 identifica como la principal amenaza, solo podrá estimarse con repeticiones en trabajos distintos.

3.3.3 Contraste del presupuesto de memoria

Los registros de vLLM permiten contrastar el presupuesto analítico de la Sección 2.7.2. Con Qwen3-8B, los pesos ocuparon 15,27 GiB, frente a los 15,3 GiB de la Ecuación 12. Con `gpu_memory_utilization` de 0,90, vLLM reservó 16,32 GiB para el KV cache, equivalentes a 118 832 tokens, y 2,44 GiB para los grafos CUDA. El cociente entre ambas cifras del KV cache es de 147 456 bytes por token, que coincide exactamente con la Ecuación 15.

El contraste revela que la configuración más exigente de la ficha no cabe en la GPU ni siquiera con el modelo piloto. Con 8192 tokens por solicitud, el KV cache disponible admite 14,51 secuencias simultáneas, según el propio registro de vLLM, y no las 16 que exige la ficha. En las ejecuciones posteriores, la reserva subió a 16,85 GiB y 122 672 tokens, que admiten 14,97 secuencias, cifra que sigue por debajo de 16. El presupuesto de la Ecuación 17 era correcto en sus términos, pero omitía los 2,44 GiB de los grafos CUDA y el 10 % de la memoria que vLLM deja sin reservar.

Con Qwen3-14B, el ensayo de la iteración 6 aporta el primer término del presupuesto: los pesos ocuparon 27,52 GiB en la GPU, frente a los 27,6 GiB de la Ecuación 12. El arranque terminó antes de que vLLM reservara el KV cache, de

modo que la estimación de unos 40 000 tokens de la Ecuación 18 sigue sin contrastarse. Ese contraste es la primera medición que producirá la fase 2.

Este resultado cierra el primero de los huecos señalados en la Sección 2.3. La combinación de 8192 tokens y concurrencia 16 solo sería ejecutable aumentando `gpu_memory_utilization`, desactivando los grafos CUDA con `--enforce-eager` o aceptando que el planificador ponga solicitudes en espera. Cualquiera de las tres opciones altera lo que se mide, y por esa razón E1 limita la entrada a 7680 tokens con una sola solicitud, y E2 limita la entrada a 1024 tokens.

3.3.4 Validez de las mediciones con Ollama

La medición de `qwen3-coder:30b` cumple las condiciones de validez del protocolo: el modelo se cargó completo en la GPU, que pasó de 0 a 18 951 MiB ocupados; las quince solicitudes medidas terminaron sin fallos y generaron los 128 tokens pedidos; y la dispersión fue baja. La desviación típica del TPOT reportado por el servidor no superó 0,07 ms en ningún punto, y la del prefill fue de 8,6 ms, 0,3 ms y 1,7 ms para las tres longitudes de entrada. La carga del modelo, cronometrada aparte, tardó 85,8 s, prácticamente todo leído desde el NFS.

La medición de `nemotron-3-nano:30b`, lanzada a continuación sin reiniciar el servidor, se invalidó. Su registro muestra que Ollama no pudo refrescar la memoria libre de la GPU y usó un dato anterior, de 20,4 GiB libres, en el que aún figuraba el modelo previo. Con ese dato colocó solo 43 de las 53 capas en la GPU, 17,9 GiB, y 4,7 GiB en la CPU. La carga tardó 226,8 s, y la primera solicitud de calentamiento no había terminado 54 minutos después, con la GPU al 0 % de utilización y un load average cercano a 790. Una medición con parte del modelo en la CPU no caracteriza la GPU, y por eso se descarta completa y no solo sus valores extremos. La regla derivada, reiniciar el servidor al cambiar de modelo, es el requerimiento RF8.

La triangulación entre cliente y servidor detectó, por último, una discrepancia. El TTFT medido por el cliente superó al prefill reportado por Ollama en 0,96 a 2,41 s por solicitud, y en cada una de las quince solicitudes esa diferencia coincidió, con un margen de 10 ms, con el campo `load_duration` de la respuesta, pese a que el modelo seguía cargado y la memoria de la GPU no cambió. El resto de la diferencia, unos milisegundos, corresponde a la red local y al cliente. El costo se origina, por tanto, en el servidor y antes del prefill, y su causa no está diagnosticada; una hipótesis compatible con los registros, pero no verificada, es la sobrecarga de CPU del nodo. En consecuencia, el TTFT de Ollama no se usa como medida del prefill, y en su lugar se usa la duración que reporta el servidor.

3.4 Resultados

Los resultados disponibles proceden de dos fuentes: el barrido sin servidor del piloto con Qwen3-8B, con lote $B = 1$, y la medición de contraste de `qwen3-coder:30b` con Ollama. Los primeros se calculan con el método de diferencia de latencias de la Sección 2.4.1 y se contrastan con los techos teóricos de la Sección 2.7.2; los segundos se usan solo para contrastar tendencias.

3.4.1 Fase de prefill con Qwen3-8B

La Tabla 24 aplica las Ecuaciones 7 y 8 a las dos longitudes de entrada medidas. La columna de MFU (*model FLOPs utilization*) expresa el throughput de prefill como fracción del techo de cómputo [22], con la Ecuación 19, donde P_{BF16} es el rendimiento BF16 denso de la A100 y N el número de parámetros.

$$\text{MFU} = \frac{2N \times X_{prefill}}{P_{BF16}} \tag{19}$$

Tabla 24: Latencia y throughput de prefill de Qwen3-8B con lote 1.

L_{in} **(tokens)**	$T_{prefill}$ **(ms)**	$X_{prefill}$ **(tok/s)**	**MFU (%)**
128	17,09	7 491	39,4
512	37,21	13 761	72,3

Elaborado por: José Andrés Ureta Granda
Fuente: Elaboración propia a partir del trabajo 30732

El throughput de prefill casi se duplica al cuadruplicar la entrada, mientras que la latencia solo se multiplica por 2,2. Ese comportamiento indica que una parte de $T_{prefill}$ no depende de la longitud de entrada. Si se supone una relación lineal, $T_{prefill} = a + b\,L_{in}$, los dos puntos medidos determinan la pendiente de la Ecuación 20 y una ordenada en el origen $a \approx 10{,}4$ ms.

$$b = \frac{T_{prefill}(512) - T_{prefill}(128)}{512 - 128} = \frac{37{,}21 - 17{,}09}{384}\ \text{ms} \approx 0{,}0524\ \text{ms/token} \tag{20}$$

El inverso de esa pendiente equivale a 19 085 tok/s, prácticamente igual al techo de 19 000 tok/s de la Ecuación 14. Sin embargo, esta coincidencia no permite afirmar que el prefill alcance el 100 % del techo de cómputo en el margen. Lo más probable es que el modelo lineal sea demasiado simple. Con 128 tokens, las multiplicaciones

de matrices son pequeñas y aprovechan peor los Tensor Cores que con 512, de modo que la eficiencia crece con L_{in} y la pendiente entre dos puntos queda subestimada. Además, la Ecuación 14 ignora las operaciones de la atención, que crecen con el cuadrado de la longitud.

Con dos puntos no es posible separar el costo fijo de la variación de la eficiencia. Por esa razón, el experimento E1 mide siete longitudes entre 128 y 7680 tokens. El dato robusto de este piloto es la cota inferior: con 512 tokens, el prefill ya alcanza el 72 % del techo de cómputo, lo que confirma que es una fase limitada por el cómputo.

3.4.2 Fase de decode con Qwen3-8B

La Ecuación 9 aplicada a la entrada de 128 tokens da un TPOT de 13,11 ms, equivalente a 76,3 tok/s por secuencia. Ese valor representa el 80 % del techo de 95 tok/s de la Ecuación 13. La Ecuación 21 expresa la misma relación como MBU (*model bandwidth utilization*), la fracción del ancho de banda de la memoria que se aprovecha al leer los pesos en cada paso [23].

$$\text{MBU} = \frac{M_{pesos}/\text{TPOT}}{BW_{HBM}} = \frac{16{,}4\ \text{GB}/13{,}11\ \text{ms}}{1555\ \text{GB/s}} = \frac{1251\ \text{GB/s}}{1555\ \text{GB/s}} \approx 80\,\% \qquad (21)$$

Un MBU del 80 % indica que el decode con lote 1 está limitado por el ancho de banda de la memoria y opera cerca de su techo. El 20 % restante corresponde a la lectura del KV cache, que la Ecuación 13 ignora, y al sobrecosto por paso del motor, que no se puede separar con estos datos. La desviación típica de 0,31 ms sobre 1695 ms de la Tabla 23 implica que el TPOT se conoce con una incertidumbre inferior a 0,01 ms.

El TPOT de 512 tokens de entrada no se obtuvo porque la ejecución con $L_{out} = 129$ no se completó. Por tanto, el piloto no permite observar cómo crece el TPOT con el contexto, que es el efecto esperado de la lectura del KV cache. Si el MBU se mantiene en el mismo nivel con Qwen3-14B, la Ecuación 13 anticipa para el modelo mayor un TPOT cercano a 24 ms con lote 1, una predicción que la campaña permitirá verificar.

3.4.3 Contraste entre fases

Los dos resultados anteriores cuantifican la asimetría que motiva el trabajo. Procesar los 128 tokens del prompt costó 17,09 ms, mientras que generar un único token costó 13,11 ms. En la práctica, el prefill procesó 128 tokens en un tiempo

apenas un 30 % mayor que el que el decode necesita para generar uno solo. La Tabla 25 resume ambas fases frente a sus techos.

Tabla 25: Comparación de las fases de Qwen3-8B frente a sus techos teóricos ($B = 1$).

Fase	Recurso limitante	Medido (tok/s)	Techo (tok/s)	Utilización
Prefill	Cómputo	7 491 ($L_{in} = 128$) 13 761 ($L_{in} = 512$)	19 000	MFU 39 % MFU 72 %
Decode	Ancho de banda	76,3	95	MBU 80 %

Elaborado por: José Andrés Ureta Granda

Fuente: Elaboración propia a partir del trabajo 30732 y de la Sección 2.7.2

Las dos fases difieren en dos órdenes de magnitud en throughput por secuencia, como anticipaba el análisis de la Sección 2.7.2. Con lote 1, el decode aprovecha una fracción mayor de su recurso limitante que el prefill de entradas cortas. Sin embargo, su throughput absoluto es 98 veces menor con 128 tokens y 180 veces menor con 512.

3.4.4 Telemetría de la GPU

En el piloto, la GPU ocupó como máximo 36 799 MiB, el 90 % de su memoria, conforme al valor de `gpu_memory_utilization`. En reposo, con la memoria vacía, la potencia media fue de 55,2 W. La potencia máxima, 268 W, se registró durante el calentamiento y la medición de la configuración con 129 tokens de salida, que es la única que mantuvo la GPU ocupada durante más de unos segundos.

La telemetría del trabajo 30732 no permite estimar la energía por token con la Ecuación 11. Con un muestreo cada 2 s, las mediciones de prefill, que duraron menos de medio segundo, no contienen ninguna muestra propia, y la de decode contiene unas ocho. Esta limitación motivó la reducción del periodo a 1 s en los scripts posteriores. En E2, cuyos puntos duran varios minutos, ese periodo bastará para integrar la energía, pero en E1 seguirá siendo insuficiente.

3.4.5 Contraste con Ollama

La Tabla 26 recoge las medianas de las cinco solicitudes medidas de `qwen3-coder:30b` en cada punto. El prefill y el TPOT son los que reporta el servidor, por la razón expuesta en la Sección 3.3.4.

Elaborado por: José Andrés Ureta Granda

Tabla 26: Prefill y decode de `qwen3-coder:30b` servido con Ollama (medianas, 128 tokens de salida).

Entrada (tokens)	Prefill (ms)	Prefill (tok/s)	TPOT (ms)	Decode (tok/s)	Latencia (s)
195	100,1	1 947	8,5	117,5	2,9
717	270,2	2 654	8,7	115,1	2,8
2 800	918,0	3 050	9,1	109,7	4,0

Fuente: Trabajo 33107 en `compute-0-1`, corrida del 11 de noviembre de 2026

Estas cifras no son comparables en valor absoluto con las de vLLM. El modelo es de tipo Mixture-of-Experts, con 30,5 mil millones de parámetros totales de los que solo una fracción se activa en cada token, y se sirve cuantizado a 4 bits en formato GGUF (`Q4_K_M`); ni el número de parámetros que se leen por paso ni los bytes por parámetro coinciden con los de un modelo denso en BF16. Por eso su TPOT, 8,5 ms, es menor que el de Qwen3-8B pese a tener casi cuatro veces más parámetros totales.

Las tendencias, en cambio, sí son comparables, y confirman dos comportamientos del piloto. La primera es que el throughput de prefill crece con la entrada, de 1947 a 3050 tok/s entre 195 y 2800 tokens, igual que con Qwen3-8B crecía entre 128 y 512: el prefill aprovecha mejor la GPU cuanto más largo es el prompt. La segunda aporta lo que el piloto no pudo medir: el TPOT crece con el contexto, de 8,5 a 9,1 ms, un 7 % al multiplicar la entrada por 14, lo que es coherente con un decode dominado por la lectura de los pesos en el que la lectura del KV cache añade un término pequeño y creciente. Ambas observaciones deberán verificarse con Qwen3-14B en vLLM.

3.4.6 Estado de la campaña con Qwen3-14B y alcance de los resultados

La campaña con Qwen3-14B no ha producido todavía mediciones de rendimiento. Lo implementado y verificado para ella es lo siguiente: la revisión del modelo está descargada y congelada; el modelo carga en la GPU con el entorno de la Sección 2.6.2 y ocupa 27,52 GiB; los experimentos E1 y E2 están configurados; y el arranque, que es el obstáculo, está diagnosticado y cronometrado por fases. Lo que falta es completar la iteración 7, es decir, reducir el costo de llevar el entorno al disco local y medir el arranque desde allí con `fase2_arranque.sh`, y después ejecutar E1 y E2.

Los resultados de este capítulo validan el método de diferencia de latencias, los techos teóricos y los dos clientes de medición, pero tienen un alcance limitado. Los de Qwen3-8B proceden de un único trabajo en un único nodo, con lote 1, dos

longitudes de entrada y una de salida, y solo aplican el primero de los tres métodos de triangulación. Los de Ollama aplican el segundo y el tercero, pero sobre un modelo y una representación numérica distintos de los del estudio. La aplicación de los tres métodos al mismo modelo, y con ella la respuesta a la pregunta del trabajo, corresponde a la campaña con Qwen3-14B.

3.5 Cronograma y Presupuesto

La Tabla 27 recoge el cronograma ejecutado hasta la fecha, ordenado según las iteraciones de la Tabla 7, y las actividades pendientes.

Tabla 27: Cronograma ejecutado y pendiente.

Periodo	**Actividad**	**Estado**
8–20 oct 2026	Especificación, modelo de datos y base del banco de pruebas	Completada
21–24 oct	Instalación del entorno y parche de glibc (iteración 1)	Completada
24 oct	Piloto con Qwen3-8B, trabajo 30732 (iteración 2)	Completada en parte
25–27 oct	Cliente `tfgbench` y validación en MIG (iteración 3)	Completada
27–29 oct	Campaña con Qwen3-8B y cambio a Qwen3-14B (iteraciones 4 y 5)	Sin medición
2 nov	Ensayo del arranque de Qwen3-14B desde el NFS (iteración 6)	Completada
8–11 nov	Copia del entorno a `/scratch` (iteración 7)	En curso
11 nov	Contraste con Ollama (iteración 8)	Completada
Por definir	Arranque desde `/scratch`, E1 y E2 con Qwen3-14B	Pendiente
Por definir	Análisis, modelo predictivo y cierre de la memoria	Pendiente

Elaborado por: José Andrés Ureta Granda

Fuente: Registros de los trabajos de Slurm y `ESTADO_DEL_PROYECTO.md`

[Presentar el detalle de los costos asociados a hardware, licencias, servicios u otros recursos.]

CAPÍTULO IV

CONCLUSIONES Y RECOMENDACIONES

4.1 Conclusiones

[Redactar conclusiones basadas en los resultados obtenidos y en el cumplimiento de los objetivos planteados.]

4.2 Recomendaciones

[Proponer mejoras, optimizaciones o ampliaciones del proyecto, fundamentadas en el análisis realizado.]

4.3 Trabajos Futuros

[Plantear líneas de desarrollo o investigación que puedan derivarse del proyecto implementado.]

[Incluya todas las referencias bibliográficas utilizadas en el documento. Deben estar citadas según normas IEEE y gestionadas mediante un gestor bibliográfico como Mendeley o Zotero.]

BIBLIOGRAFÍA

[1] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, 2024. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

[2] Qwen Team, "Qwen3 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[3] NVIDIA Corporation, "NVIDIA A100 tensor core GPU datasheet," https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf, 2021, consultado el 16 de octubre de 2026.

[4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. Koblenz, Germany: ACM, 2023, arXiv:2309.06180.

[5] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou *et al.*, "A survey of large language models," 2023. [Online]. Available: https://arxiv.org/abs/2303.18223

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. Curran Associates, Inc., 2017, arXiv:1706.03762.

[7] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*.

Santa Clara, CA: USENIX Association, 2024. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/agrawal

[8] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-Based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, 2022, pp. 521–538. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/yu

[9] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-Awareness," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022, arXiv:2205.14135.

[10] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proceedings of Machine Learning and Systems 6 (MLSys 2024)*, Santa Clara, CA, 2024, best Paper Award. arXiv:2306.00978.

[11] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, and Y. He, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022.

[12] Z. Li, L. Zheng, Y. Zhong, V. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen, H. Zhang, J. E. Gonzalez, and I. Stoica, "AlpaServe: Statistical multiplexing with model parallelism for deep learning serving," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. Boston, MA: USENIX Association, Jul. 2023, pp. 663–679.

[13] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *Proceedings of Machine Learning and Systems*, vol. 5, 2023.

[14] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, and W. Lin, "Llumnix: Dynamic scheduling for large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 173–191.

[15] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution

of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[16] Hugging Face, "Text Generation Inference," Repositorio de GitHub. [En línea]. Disponible en: https://github.com/huggingface/text-generation-inference, 2023, archivado el 21 de marzo de 2026. Consultado el 29 de octubre de 2026.

[17] NVIDIA Corporation, "TensorRT-LLM: Overview," Documentación oficial. [En línea]. Disponible en: https://nvidia.github.io/TensorRT-LLM/overview.html, 2023, consultado el 29 de octubre de 2026.

[18] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative LLM inference using phase splitting," in *ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024.

[19] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[20] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, 2023, pp. 4895–4901. [Online]. Available: https://aclanthology.org/2023.emnlp-main.298/

[21] D. Kalamkar, D. Mudigere, N. Mellempudi, D. Das, K. Banerjee, S. Avancha *et al.*, "A study of BFLOAT16 for deep learning training," 2019. [Online]. Available: https://arxiv.org/abs/1905.12322

[22] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts *et al.*, "PaLM: Scaling language modeling with pathways," *Journal of Machine Learning Research*, vol. 24, no. 240, pp. 1–113, 2023. [Online]. Available: https://www.jmlr.org/papers/v24/22-1144.html

[23] M. Agarwal, A. Qureshi, N. Sardana, L. Li, J. Quevedo, and D. Khudia, "LLM inference performance engineering: Best practices," Databricks Blog. [En línea]. Disponible en: https://www.databricks.com/blog/llm-inference-performance-engineering-best-practices, Oct. 2023, consultado el 24 de octubre de 2026.

[24] A. B. Yoo, M. A. Jette, and M. Grondona, "SLURM: Simple Linux utility for resource management," in *Job Scheduling Strategies for Parallel Processing (JSSPP 2003)*, ser. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2003, pp. 44–60.

[25] N. J. Smith and T. Kluyver, "PEP 600 – future 'manylinux' platform tags for portable Linux built distributions," Python Enhancement Proposals. [En línea]. Disponible en: https://peps.python.org/pep-0600/, 2019, consultado el 24 de octubre de 2026.

[26] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024, arXiv:2307.08691.

[27] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024, pp. 68 658–68 685.

GLOSARIO

[Defina los términos técnicos relevantes utilizados en el documento.]

Término o sigla 1 *[Definición técnica breve y clara.]*

Término o sigla 2 *[Definición técnica breve y clara.]*

ANEXO A

TÍTULO DEL ANEXO A

[Incluya una descripción breve del contenido del anexo. Ejemplo: documentación técnica, diagramas, configuraciones, evidencias o instrumentos aplicados.]

- *[Documentación técnica.]*
- *[Manuales de usuario y administración.]*
- *[Evidencias de pruebas.]*
- *[Autorizaciones y validaciones de la entidad receptora.]*
- *[Repositorio de versionamiento (enlace).]*
- *[Formatos de entrevistas y encuestas aplicadas.]*